

Sincronizzazione

Luca Spalazzi

spalazzi@dii.univpm.it

www.univpm.it/luca.spalazzi



- Modello a memoria condivisa
 - Corsa critica, Sezione critica,
 - Spinlock, Semafori, Monitor
 - Problemi classici di sincronizzazione
 - Implementazione su macchine mono- e multi-processore
- Modello a scambio di messaggi



Sincronizzazione

CORSA CRITICA E SEZIONE CRITICA

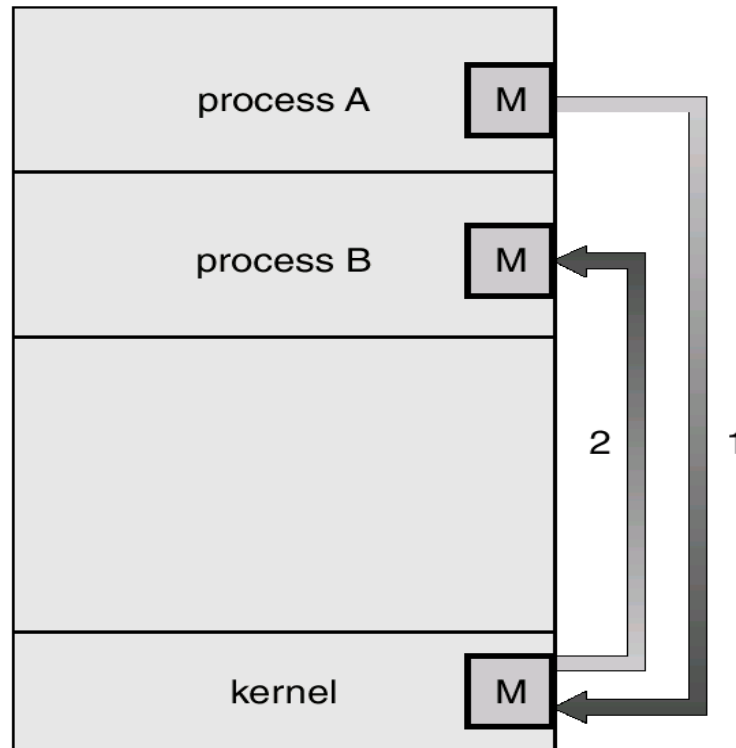
- **Processi Indipendenti** – L' esecuzione di un processo NON dipende dall' esecuzione degli altri processi.
- **Processi Cooperanti** – L' esecuzione di un processo dipende dall' esecuzione degli altri processi.
- Vantaggi della cooperazione tra processi
 - Condivisione delle informazioni
 - Accelerazione dell'elaborazione
 - Modularità
 - Convenienza



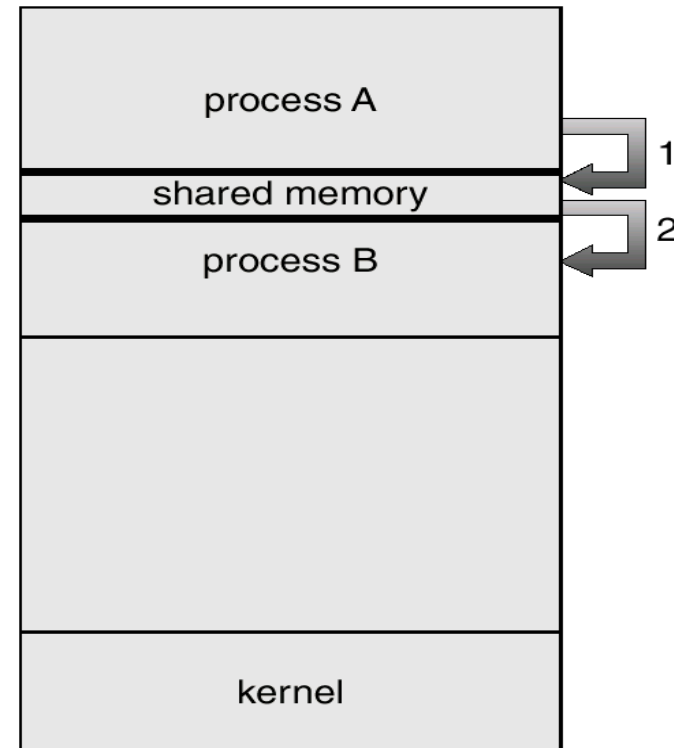
Interprocess Communication (IPC)

- Per permettere la cooperazione tra più processi e/o thread servono meccanismi per gestire la comunicazione e la sincronizzazione.
 - **Modello a memoria condivisa**
 - **Modello a scambio di messaggi**

Comunicazione tra processi



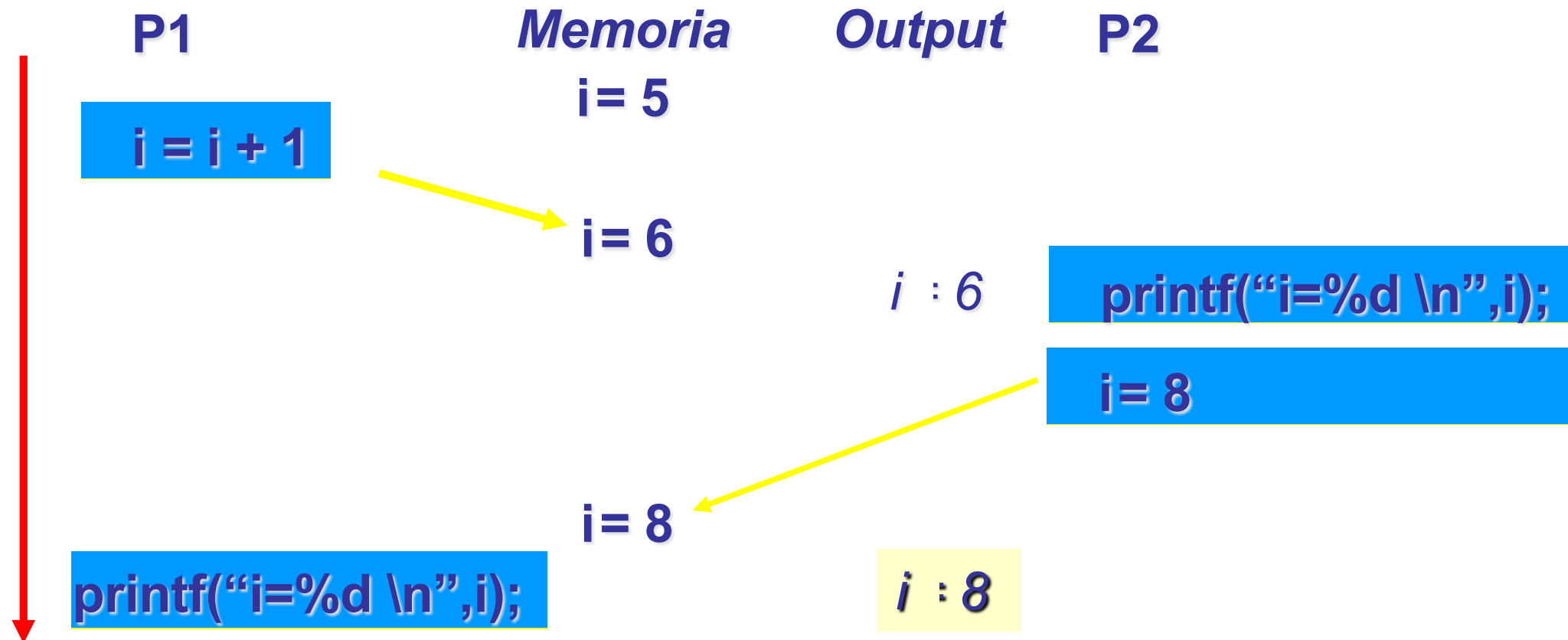
Scambio di Messaggi



Memoria Condivisa

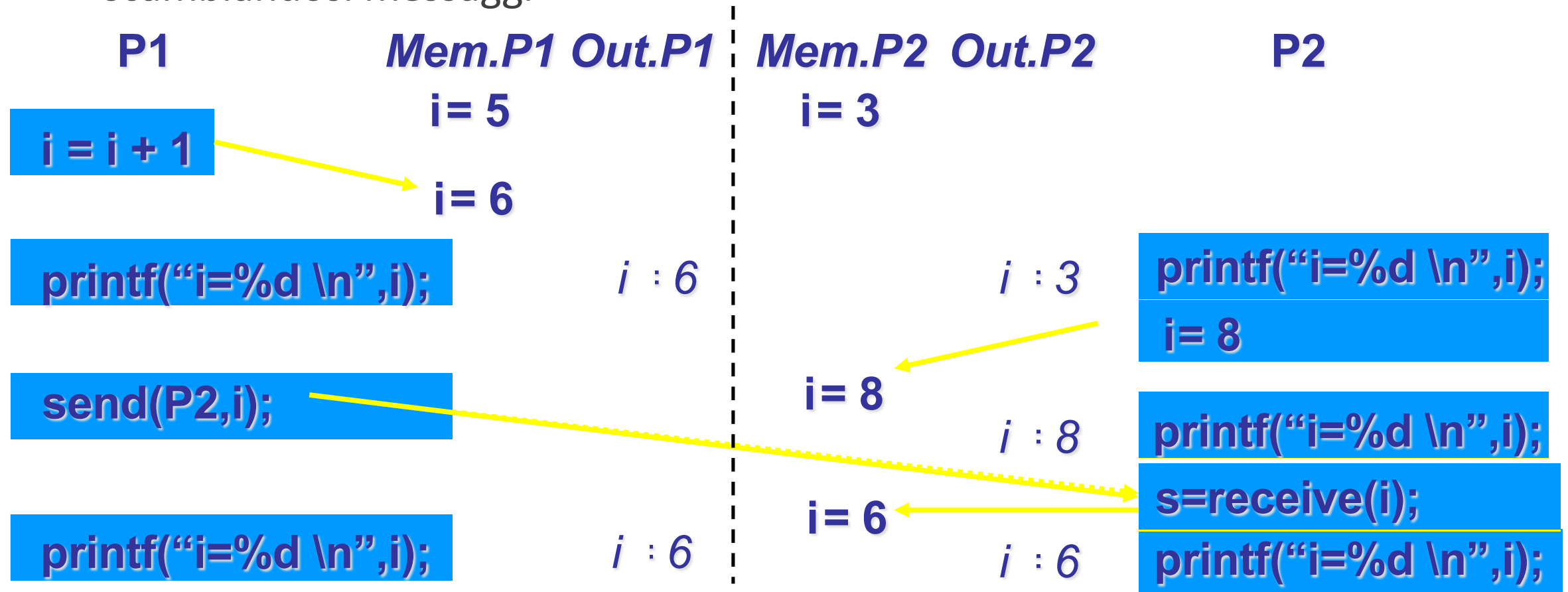
Modello a Memoria Condivisa

- Se un processo modifica una variabile condivisa, anche gli altri processi vedranno la modifica



Modello a Scambio di Messaggi

- processi non condividono variabili in memoria, ma possono interagire scambiandosi messaggi



```

1. public class CCounter implements Runnable {
2.     private int counter = 0;

    // increment counter by one
3.     public void run() {
4.         int tmp = getCounter();
5.         setCounter(tmp + 1);
6.     }

```

...
}

t	u	shared
pc _t =4; tmp _t =⊥	pc _u =4; tmp _u =⊥	counter=0
pc _t =5; tmp _t =0	pc _u =4; tmp _u =⊥	counter=0
pc _t =6; tmp _t =0	pc _u =4; tmp _u =⊥	counter=1
done	pc _u =4; tmp _u =⊥	counter=1
done	pc _u =5; tmp _u =1	counter=1
done	pc _u =6; tmp _u =1	counter=2
done	done	counter=2

Risultato corretto

```
1. public class CCounter implements Runnable {
2.     private int counter = 0;
```

```
    // increment counter by one
```

```
3.     public void run() {
4.         int tmp = getCounter();
5.         setCounter(tmp + 1);
6.     }
```

```
...
}
```

t	u	shared
pc _t =4; tmp _t =⊥	pc _u =4; tmp _u =⊥	counter=0
pc _t =5; tmp _t =0	pc _u =4; tmp _u =⊥	counter=0
pc _t =5; tmp _t =0	pc _u =5; tmp _u =0	counter=0
pc _t =5; tmp _t =0	pc _u =6; tmp _u =0	counter=1
pc _t =6; tmp _t =0	done	counter=1
done	done	counter=1

Risultato errato



- **Corsa critica (race condition):** Si ha quando più processi possono accedere e manipolare dei dati condivisi e il valore finale dei dati dipende dall'ordine degli accessi.
- Per prevenire la corsa critica i processi devono essere sincronizzati tra loro.

Soluzione: Sezione Critica

- n processi competono per accedere ad una data risorsa (per es. un'area di memoria condivisa)
- Ogni processo ha un segmento di codice, chiamato **sezione critica**, che contiene le istruzioni di accesso e manipolazione della risorsa condivisa.
- Soluzione
 - assicurare che quando un processo è in esecuzione nella propria sezione critica, nessun altro processo possa essere in esecuzione nella propria sezione critica.

Sezione Critica: Requisiti e Assunzioni

Requisiti

1. **Mutua Esclusione.** Se il processo P_i è in esecuzione nella propria sezione critica, nessun altro processo può essere in esecuzione nella propria sezione critica.
2. **Progresso.** Se nessun processo è nella sua sezione critica e ci sono alcuni processi che desiderano entrare nella propria sezione critica, allora prima o poi uno di quei processi entra in sezione critica (**assenza di deadlock**).
3. **Attesa limitata.** Se esiste un processo in attesa di entrare nella propria sezione critica, allora prima o poi entra in sezione critica (**assenza di starvation**).

Assunzioni

- Si deve assumere che ogni processo ha un tempo di esecuzione non nullo
- Nessuna assunzione deve essere fatta circa la velocità relativa dei processi.

Sezione Critica: Struttura generale

```
// continuously  
while (true) {  
    entry section  
    critical section {  
        // access shared data  
    }  
    exit section  
    // reminder section  
}
```

I processi possono condividere alcune variabili per sincronizzare le loro azioni.

Sezione Critica: Il contatore rivisitato

```
1. public class CCounter implements Runnable {  
2.     private int counter = 0;  
  
    // increment counter by one  
3.     public void run() {  
4.         entry section;  
5.         int tmp = getCounter();  
6.         setCounter(tmp + 1);  
7.         exit section;  
8.         return;  
6.     }  
...  
}
```

Ogni thread modifica
counter in modo
mutuamente esclusivo

- Come realizzare i protocolli di ingresso e uscita dalla sezione critica?
- Esistono due tipi di protocolli
 - **Spinlock** – basati sul concetto di mettere in attesa attiva un processo/thread fino a quando la risorsa non si libera
 - **Context switch** – basati sul concetto di sospendere un processo/thread, lasciando quindi la CPU disponibile agli altri, fino a quando la risorsa non si libera



Sincronizzazione **SPINLOCK**

Algoritmo con SpinLock

Viene usata una struttura dati condivisa sulla quale possono essere eseguite due operazioni: `lock()`, `unlock()`

Processo P_i :

```
1. while (true) {  
2.     lock();  
3.     critical section  
4.     unlock();  
5.     remainder section  
6. };
```

```
interface SpinLock {  
    void lock();    // acquire lock  
    void unlock(); // release lock  
}
```

- Dobbiamo trovare gli algoritmi per implementare l'interfaccia

Sezione Critica – 2 Processi/Thread

Algoritmo 1

- Variabili condivise:

- `int turn = 0;` $\Rightarrow$ valore iniziale
- `turn = i` $\Rightarrow P_i$ può entrare nella sua sezione critica

- Processo P_i

```
1. while (true) {  
2.     while (turn != i) ;  
3.     critical section  
4.     turn = i;  
5.     remainder section  
6. };
```

Soddisfa la mutua esclusione,
l'assenza di deadlock, ma non
il progresso (in generale), né
l'attesa limitata

Sezione Critica – 2 Processi/Thread

Algoritmo 2

- Variabili condivise:
 - `boolean flag[] = {false , false};` $\Rightarrow$ valore iniziale
 - `flag [i] = true` $\Rightarrow P_i$ pronto ad entrare nella sua sezione critica

- Processo P_i

```
1. while (true) {  
2.     flag[i] = true;  
3.     while (flag[j]) ;  
4.     critical section  
5.     flag[i] = false;  
6.     reminder section  
7. };
```

Soddisfa la mutua esclusione,
ma non il progresso

Sezione Critica – 2 Processi/Thread

Algoritmo di Peterson

- Combina i due algoritmi precedenti:

— `boolean flag[] = {false , false};` `int turn = 0;`

- Processo P_i

```
1. while (true) {  
2.     flag[i] = true;                    lock()  
3.     turn = j;  
4.     while (flag[j] && turn==j) ;  
5.     critical section  
6.     flag[i] = false;                unlock()  
7.     reminder section  
8. };
```

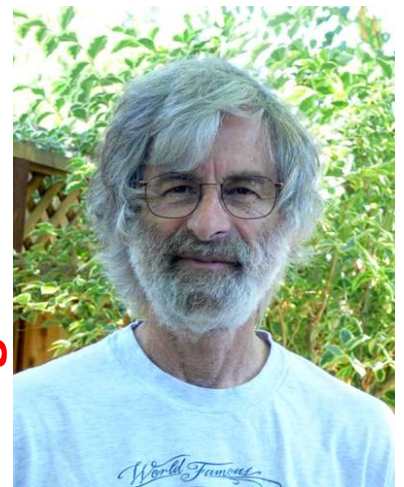
Soddisfa mutua esclusione,
progresso e attesa limitata

Sezione Critica – n Processi/Thread

Algoritmo del fornaio di Lamport (L. B. Lamport's Bakery algorithm)

- Prima di entrare nella sua regione critica, ad ogni processo viene assegnato un numero. Il processo con il numero più piccolo entra nella sua sezione critica.
- Se due processi P_i e P_j ricevono lo stesso numero, se $i < j$, allora P_i è servito per primo; altrimenti è P_j ad essere servito.
- Vengono sempre generati numeri in ordine crescente di enumerazione,
- Possono esserci processi con lo stesso numero
 - ovvero 1,2,3,3,3,3,4,5...

Turing Award 2013



Algoritmo del fornaio di Lamport (Lamport's Bakery algorithm)

■ Variabili condivise

```
public static final int numberOfThreads = 5;  
private static volatile boolean[] choosing = new boolean[numberOfThreads];  
private static volatile int[] ticket = new int[numberOfThreads];
```



Finora abbiamo assunto implicitamente che i thread eseguano le istruzioni nella sequenza che gli abbiamo dato. Ciò non è garantito da Java e dalla maggior parte dei linguaggi di programmazione quando i thread accedono ai campi condivisi.

- i compilatori possono riordinare le istruzioni in base all'analisi statica, che non conosce i thread
- i processori possono ritardare l'effetto delle scritture in memoria quando la cache è impegnata

Algoritmo del fornaio di Lamport (Lamport's Bakery algorithm)

- Variabili condivise

```
public static final int numberOfThreads = 5;  
private static volatile boolean[] choosing = new boolean[numberOfThreads];  
private static volatile int[] ticket = new int[numberOfThreads];
```



Accedere ad un campo (attributo) dichiarato come volatile forza la sincronizzazione e quindi impedisce qualsiasi ottimizzazione che comporti il riordino delle istruzioni

Sezione Critica – n Processi/Thread

Algoritmo del fornaio di Lamport (Lamport's Bakery algorithm)

- Processo P_i

```
while (true) {
```

```
    choosing[id] = true;
    ticket[id] = findMax() + 1;
    choosing[id] = false;
    for (int j = 0; j < numberOfThreads; j++) {
        while (choosing[j]) ;
        while (ticket[j] != 0 &&
              (ticket[id] > ticket[j] || (ticket[id] == ticket[j] && id > j))) ;
    }
```

critical section

```
    ticket[id] = 0;
```

reminder section

```
};
```

Soddisfa mutua esclusione,
progresso e attesa limitata

lock()

unlock()

Sezione Critica – n Processi/Thread

- Algoritmo del fornaio di Lamport usa $\Theta(n)$ locazioni di memoria condivisa (i due array di n elementi). Si può provare che è la quantità minima di memoria richiesta per avere mutua esclusione se si usano solo operazioni di lettura e scrittura.
- Per questa ragione la sincronizzazione usando solo lettura e scrittura non è praticabile. Servono primitive più potenti:
 - operazioni atomiche
 - operazioni di sospensione e riattivazione dei processi

- Controlla e modifica il contenuto di una variabile Booleana atomicamente.

```
1.    boolean TestAndSet(boolean *target) {  
2.        boolean rv = *target;  
3.        *target = true;  
4.        return rv;  
5.    }
```




- Variabile condivisa:
boolean lock = false;

Processo Pi

```
while (true) {
```

```
    lock();
```

```
    critical section
```

```
    unlock();
```

```
    remainder section
```

```
}
```

```
enter_region:  
    TSL REGISTER,LOCK  
    CMP REGISTER,#0  
    JNE enter_region  
    RET | return to caller;
```

```
leave_region:  
    MOVE LOCK,#0  
    RET | return to caller
```

Mutua Esclusione con Test-and-Set Java

- Il package `atomic` fornisce supporto ad operazioni atomiche su singole variabili

```
package java.util.concurrent.atomic;
```

```
public class AtomicBoolean {
```

```
    AtomicBoolean(boolean initialValue);
```

```
    boolean get();
```

```
    void set(boolean newValue);
```

```
    boolean getAndSet(boolean newValue);
```

```
    ...
```

```
}
```

Metodo costruttore, crea una nuova istanza il cui valore è `initialValue`

Metodo getter, legge il valore corrente dell'istanza

Metodo setter, assegna all'istanza il valore `newValue`

Restituisce il valore corrente ed assegna all'istanza il valore `newValue`

`TestAndSet(&x)` è equivalente a `x.getAndSet(true)`

Algoritmo mutua esclusione con TSL

```
public class TASLock  
    implements Lock {  
  
    AtomicBoolean held = new AtomicBoolean(false);
```

```
    public void lock() {  
        while (held.getAndSet(true))  
            {}  
    }
```

Soddisfa la mutua esclusione,
il progresso, ma non l'attesa
limitata

```
    public void unlock() {  
        held.set(false);  
    }
```

```
}
```

■ Atomicamente:

- assegna alla variabile `value` il valore `new_value` se il valore corrente è uguale a `expected`,
- restituisce il valore originale di `value`.

```
1.  boolean CompareAndSwap(boolean *value, expected, new_value) {  
2.      boolean temp = *value;  
3.      if (*value == expected)  
4.          *value = new_value;  
5.      return temp;  
6.  }
```

Mutua Esclusione con CompareAnd Swap Java

- Il package `atomic` fornisce supporto ad operazioni atomiche su singole variabili

```
package java.util.concurrent.atomic;
```

```
public class AtomicBoolean {  
    AtomicBoolean(boolean initialValue);  
    boolean get();  
    void set(boolean newValue);  
    boolean getAndSet(boolean newValue);  
    boolean compareAndExchange(boolean expectedValue,  
                                boolean newValue)
```

Restituisce il valore corrente ed assegna all'istanza il valore `newValue` se il valore corrente è uguale a `expectedValue`

```
} CompareAndSwap(&x, a, b) è equivalente a x.compareAndExchange(a, b)
```

Mutua Esclusione con CompareAnd Swap Java

Algoritmo mutua esclusione con CompareAndSwap

```
public class TASLock  
    implements Lock {
```

```
    AtomicBoolean held = new AtomicBoolean(false);
```

```
    public void lock() {  
        while (held.compareAndExchange(false,true))  
            {}  
    }
```

```
    public void unlock() {  
        held.set(false);  
    }
```

```
}
```

Soddisfa la mutua esclusione,
il progresso, ma non l'attesa
limitata



Esercizio 1: Sezione Critica – n Processi

- Gli algoritmi precedenti sfruttano TSL e Swap per la mutua esclusione ma non garantiscono l'attesa limitata.
- Trovare un algoritmo che soddisfa tutti e tre i requisiti utilizzando TSL.



Sincronizzazione

SEMPAHORES AND MUTEX

Semafori e Mutex

- Strumento di sincronizzazione che non richiede il busy waiting si può utilizzare in problemi di sincronizzazione più complessi di quello della sezione critica.
- Semaforo S – variabile intera
- Si può accedere ad S solo attraverso due operazioni indivisibili

Semaforo Contatore $\langle \mathbb{Z}, \text{wait}, \text{signal} \rangle$

wait (S):

while $S \leq 0$ do no-op;
S--;

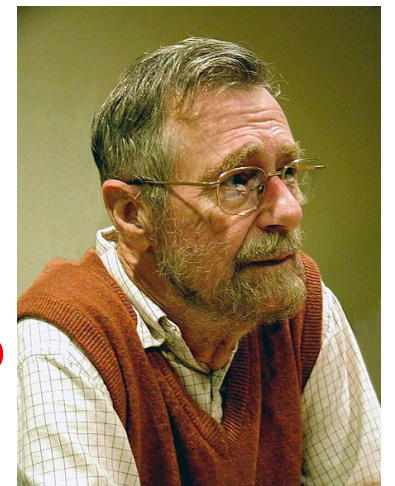
signal (S):

S++;

Semaforo Binario
(Mutex) $\langle \mathbb{B}, \text{wait}, \text{signal} \rangle$

E. W. Dijkstra

Turing Award 1972



Semafori

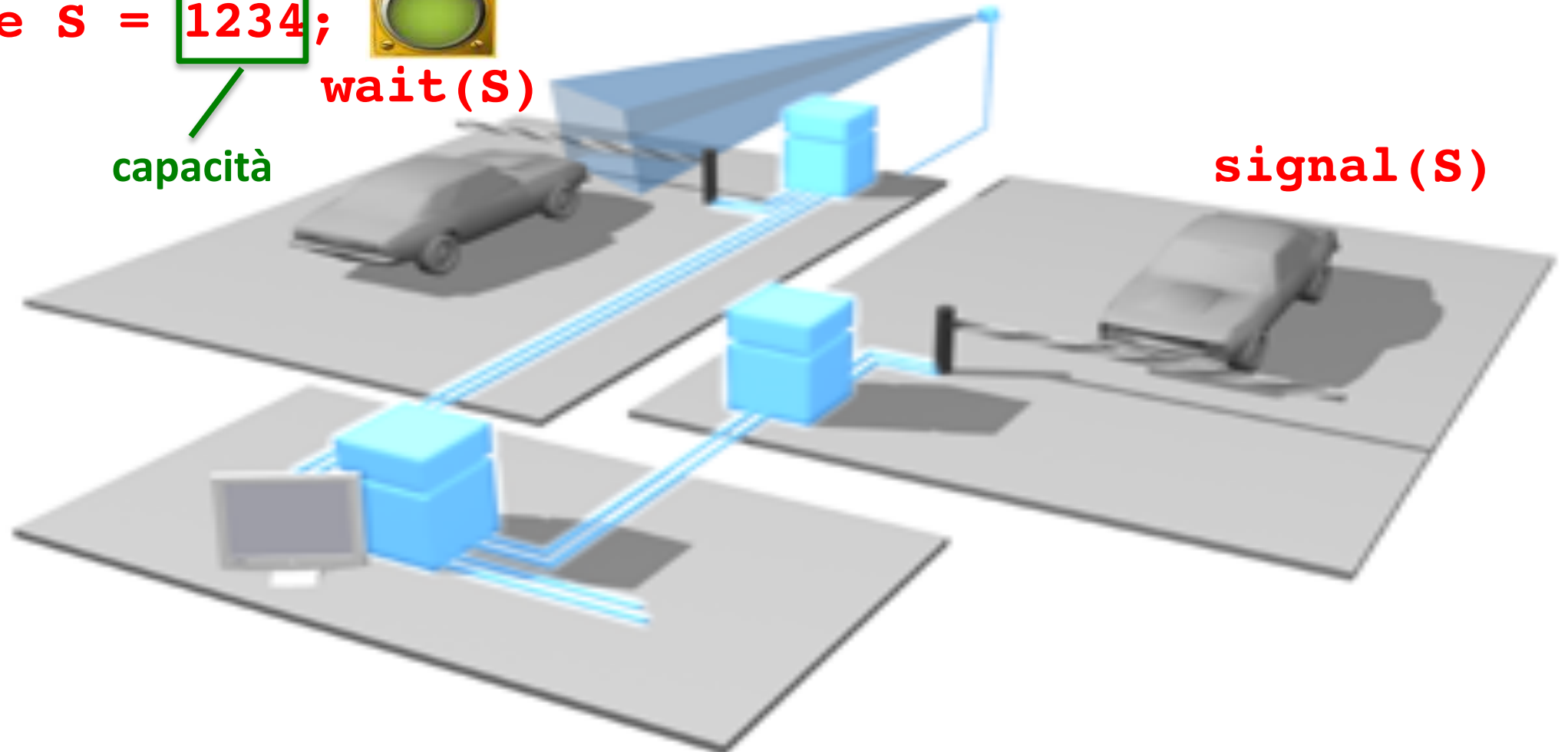
semaphore S = 1234;



wait(S)

capacità

signal(S)



Quando $S > 0$, S rappresenta il numero di posti liberi

Quando $S = 0$, non ci sono posti liberi

Algoritmo con Mutex

- Variabili condivise:

```
mutex m = true;
```

Processo P_i :

```
1. while (true) {  
2.     wait(mutex);  
3.     critical section  
4.     signal(mutex);  
5.     remainder section  
6. };
```



```
interface CounterSemaphore{  
    void cWait();  
    void cSignal();  
}  
  
interface Mutex{  
    void mWait();  
    void mSignal();  
}
```

Dobbiamo trovare gli algoritmi per implementare queste interfacce

Mutex in Java: ReentrantLock

- Java ha una interfaccia lock che di fatto corrisponde alla nostra interfaccia mutex

```
public interface Lock{  
    void lock();  
    void unlock();  
}
```

wait()

signal()

- La realizzazione di tale interfaccia più utilizzata è la classe ReentrantLock.

```
public class ReentrantLock implements Lock {  
    ReentrantLock ReentrantLock(boolean fair);  
    void lock();  
    void unlock();  
}
```

wait()

signal()

Mutex in Java: ReentrantLock

■ Soluzione

```
private final ReentrantLock l = new ReentrantLock(v);
```

```
public void m() {  
    l.lock();  
    try {  
        // critical session  
    } finally {  
        l.unlock();  
    }  
}
```

la clausola `finally` assicura che il lock venga rilasciato anche nel caso avvenga una eccezione nel blocco `try`

✓ Mutua esclusione: è garantita
✓ Progresso: garantito se non ci sono altri lock.

✗ Attesa limitata:

- se `v=false` non è garantita,
- se `v=true` favorisce l'accesso al thread in attesa da più tempo. Tuttavia la fairness del lock non garantisce la fairness dei thread: un thread può ottenere il lock più volte in successione mentre altri thread attivi non stanno procedendo e al momento non tengono il lock.

Mutex e Semafori in Java: Semaphore

- Java ha una classe Semaphore : quando è inizializzato a 1 abbiamo un semaforo binario (mutex), quando è inizializzato ad un valore maggiore di 1 abbiamo un semaforo contatore

```
public class Semaphore {
```

```
    Semaphore(int permits, boolean fair);
```

```
    void acquire();
```

wait()

```
    void release();
```

signal()

```
}
```

- permits è la capacità del semaforo,
- se `v=false` i thread vengono schedulati in modo non deterministico
- se `v=true` i thread vengono schedulati secondo un ordine FIFO

Mutex in Java: Semaphore

■ Soluzione

```
private final Semaphore s = new Semaphore(1,true);
```

```
public void m() {  
    s.acquire();  
    try {  
        // critical session  
    }  
    finally {  
        s.release()  
    }  
}
```

- ✓ Mutua esclusione: è garantita
- ✓ Progresso: garantito se non ci sono altri semafori.
- ✓ Attesa limitata: garantita



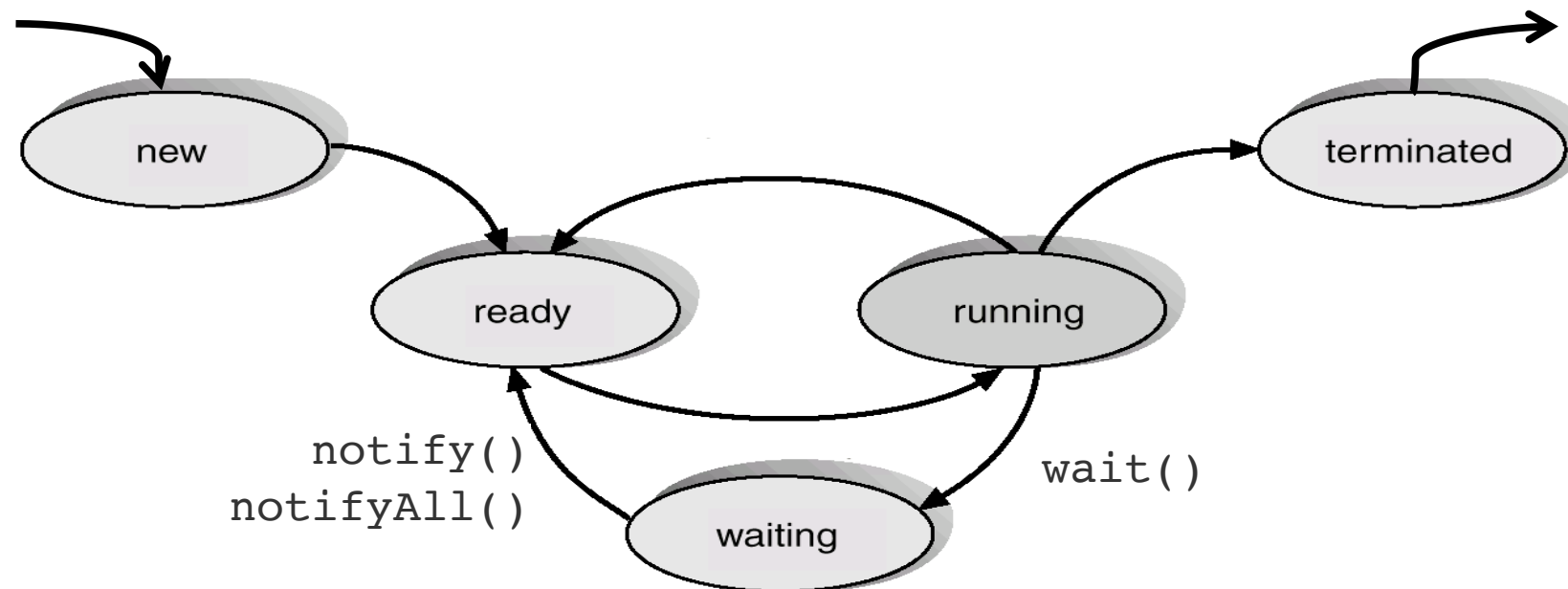
ReentrantLock vs. Semaphore

I semafori binari sono molto simili ai lock con una differenza:

- in un **lock** solo il thread che lo acquisisce (decrementa il valore) lo può rilasciare (incrementa il valore);
- in un **semaforo binario** un thread può decrementare il valore ed un altro lo può incrementare

Implementazione di CounterSemaphore

- Proviamo ad implementare una nostra realizzazione di CounterSemaphore basata sul context switch.
- Ci servono due primitive: una per bloccare un processo/thread e una per risvegliare un processo/thread
- Lavorando in Java utilizzeremo i thread e le primitive per gestirli



Implementazione di CounterSemaphore

- Proviamo ad implementare una nostra realizzazione di CounterSemaphore basata sul context switch.
- Ci servono due primitive: una per bloccare un processo/thread e una per risvegliare un processo/thread
- lavorando in Java utilizzeremo i thread e le primitive per gestirli
 - `wait()` : sospende il thread che lo esegue (interruzione sw)
 - `notify()` : sceglie in modo non deterministico un thread in stato di waiting e lo porta in stato di ready
 - `notifyAll()` : tutti i thread in stato di waiting vengono portati in stato di ready
 - devono avere effetto solo sui thread in coda sullo stesso oggetto condiviso, per questo si deve usare blocchi o metodi `synchronized`

Implementazione Debole (weak)

- Soluzione: usiamo wait() e notify()

```
class SemaphoreWeak implements Semaphore {  
    private int count;  
    public synchronized void cSignal() {  
        count = count + 1;  
        notify(); // wake up a waiting thread  
    }  
    public synchronized void cWait() throws InterruptedException {  
        while (count == 0)  
            wait(); // suspend running thread  
        count = count - 1; // now count > 0  
    }  
}
```

La wait() deve essere richiamata all'interno di un loop per evitare risvegli spuri, perché il Java non garantisce che non ce ne siano

Implementazione Forte (strong)

- Soluzione: usiamo wait() e notifyAll()

```
class SemaphoreStrong implements Semaphore {  
    public synchronized void cSignal() {  
        count = count + 1;  
        if (!blocked.isEmpty()) notifyAll();  
    }  
    public synchronized void cWait() throws InterruptedException {  
        Thread me = Thread.currentThread();  
        blocked.add(me);  
        while (!(count > 0 && blocked.element() == me))  
            wait();  
        blocked.remove();  
        count = count - 1;  
    }  
    private final Queue<Thread> blocked = new LinkedList<>();  
}
```



Esercizio 2: Due tipi di semaforo

- Semaforo Contatore – variabile intera che può variare su un dominio non limitato. I semafori visti fino ad ora sono semafori contatori.
- Semaforo Binario – variabile booleana. È più facile da implementare.
- Realizzare un semaforo contatore utilizzando i semafori binari
 - Suggerimento: utilizzare due semafori binari.



Sincronizzazione

PROBLEMI CLASSICI DI SINCRONIZZAZIONE

Un Semplice Esempio di Sincronizzazione

■ Problema

- eseguire istruzione B in P_j solo dopo l'esecuzione di A in P_i

■ Soluzione

- usare un semaforo flag inizializzato a 0
- codice:

P_i	P_j
$\vdots$	$\vdots$
A	wait(flag)
signal(flag)	B

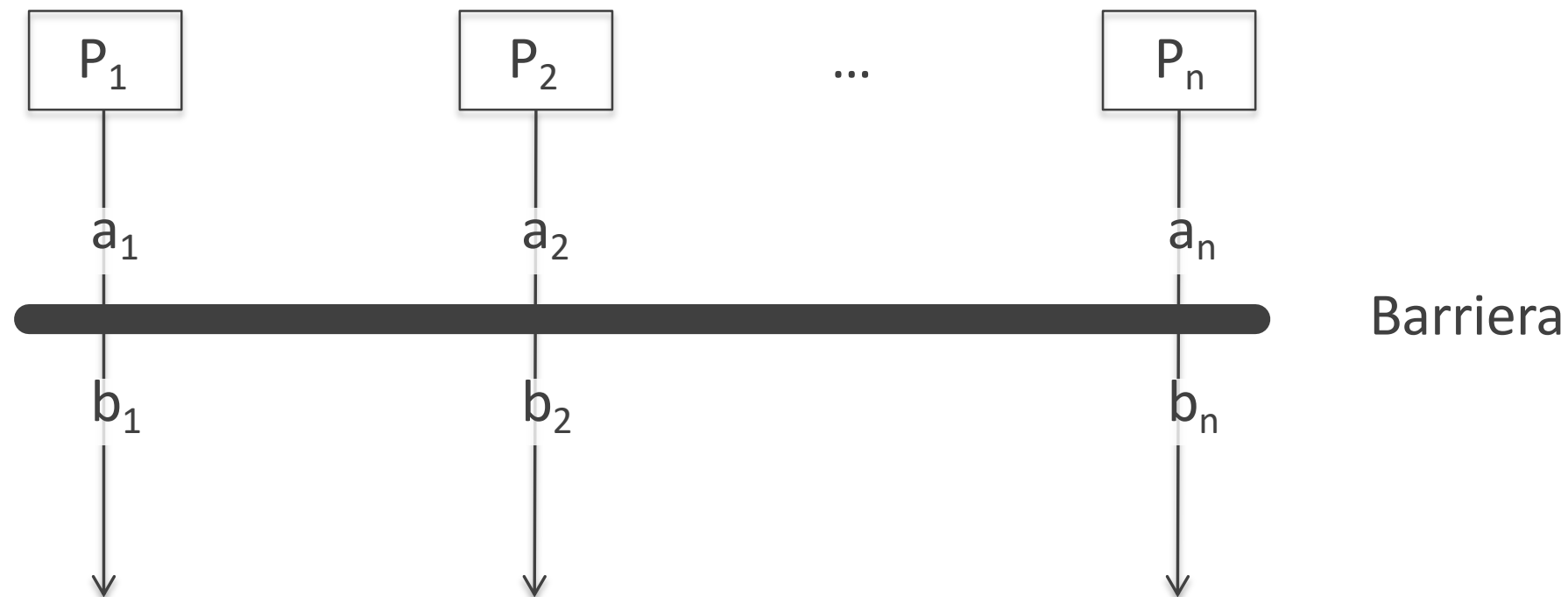


Problemi Classici di Sincronizzazione

- Barriere (rendezvous)
- Produttore-Consumatore con Buffer Limitato
- Lettori e Scrittori
- Cinque Filosofi

Barriera

- Una barriera è una forma di sincronizzazione dove esiste un punto (la barriera) nell'esecuzione di ogni processo di un certo gruppo che deve essere raggiunto da tutti i processi del gruppo prima che ognuno di loro possa proseguire nell'esecuzione



■ Soluzione per 2 processi

```
Semaphore[] done = {new Semaphore(0), new Semaphore(0)};
```

Thread t0

```
// codice prima della barriera  
done[t0].release();  
done[t1].acquire();  
// codice dopo la barriera
```

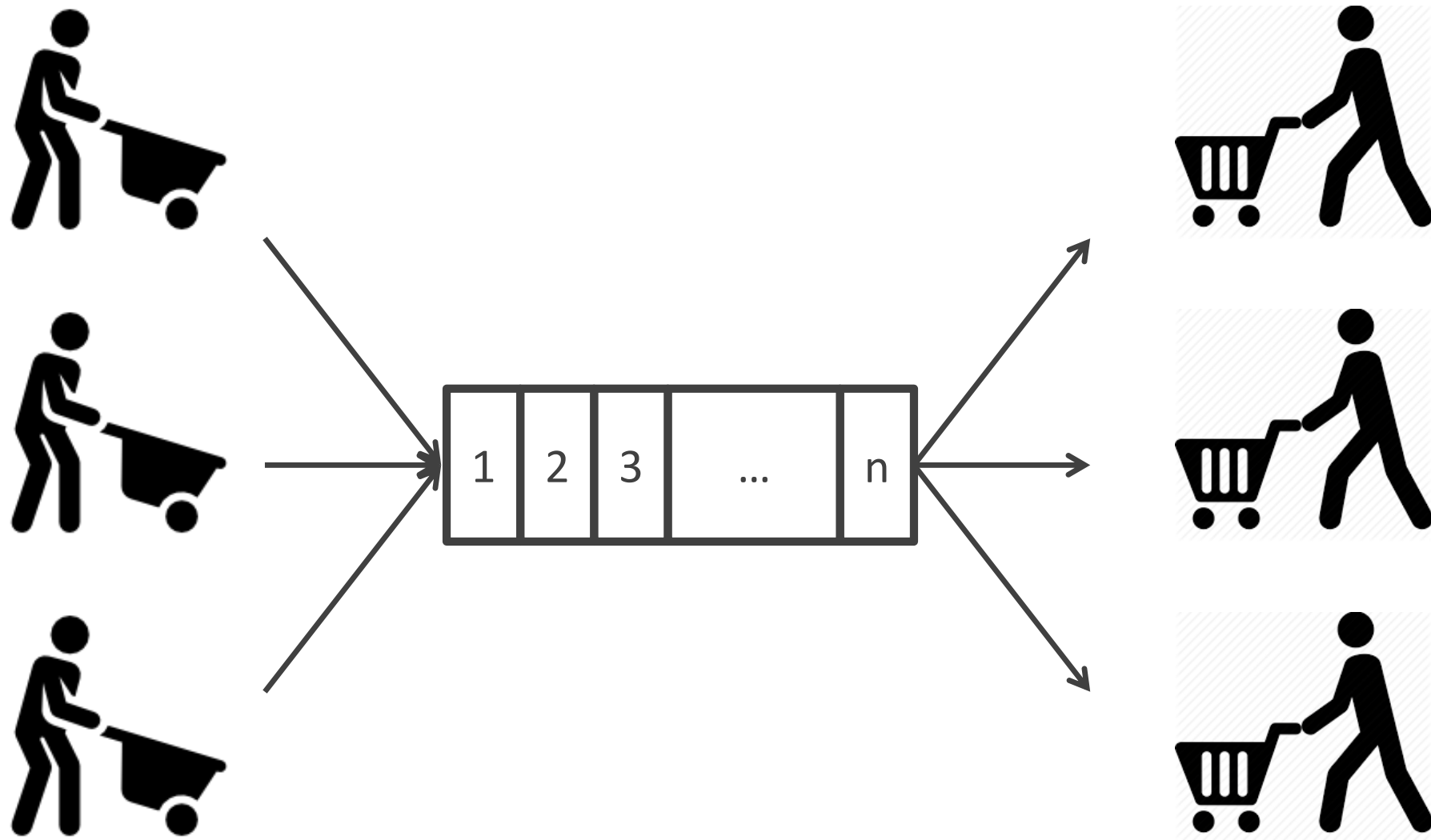
Thread t1

```
// codice prima della barriera  
done[t1].release();  
done[t0].acquire();  
// codice dopo la barriera
```



qui non si può usare un Lock perché è
acquisito da un thread e rilasciato da
un altro

Produttore e Consumatore – Buffer Limitato





Soluzione

```
public class BoundedBuffer<T> {  
    Lock lock = new Lock();  
    Semaphore nItems = new Semaphore(0);  
    Semaphore nFree = new Semaphore(N);  
    Collection storage = ...;  
    ...  
}
```

Accesso al buffer in mutua
esclusione

n. item nel buffer

n. posizioni libere nel buffer

Il buffer vero e proprio

Produttore e Consumatore – Buffer Limitato

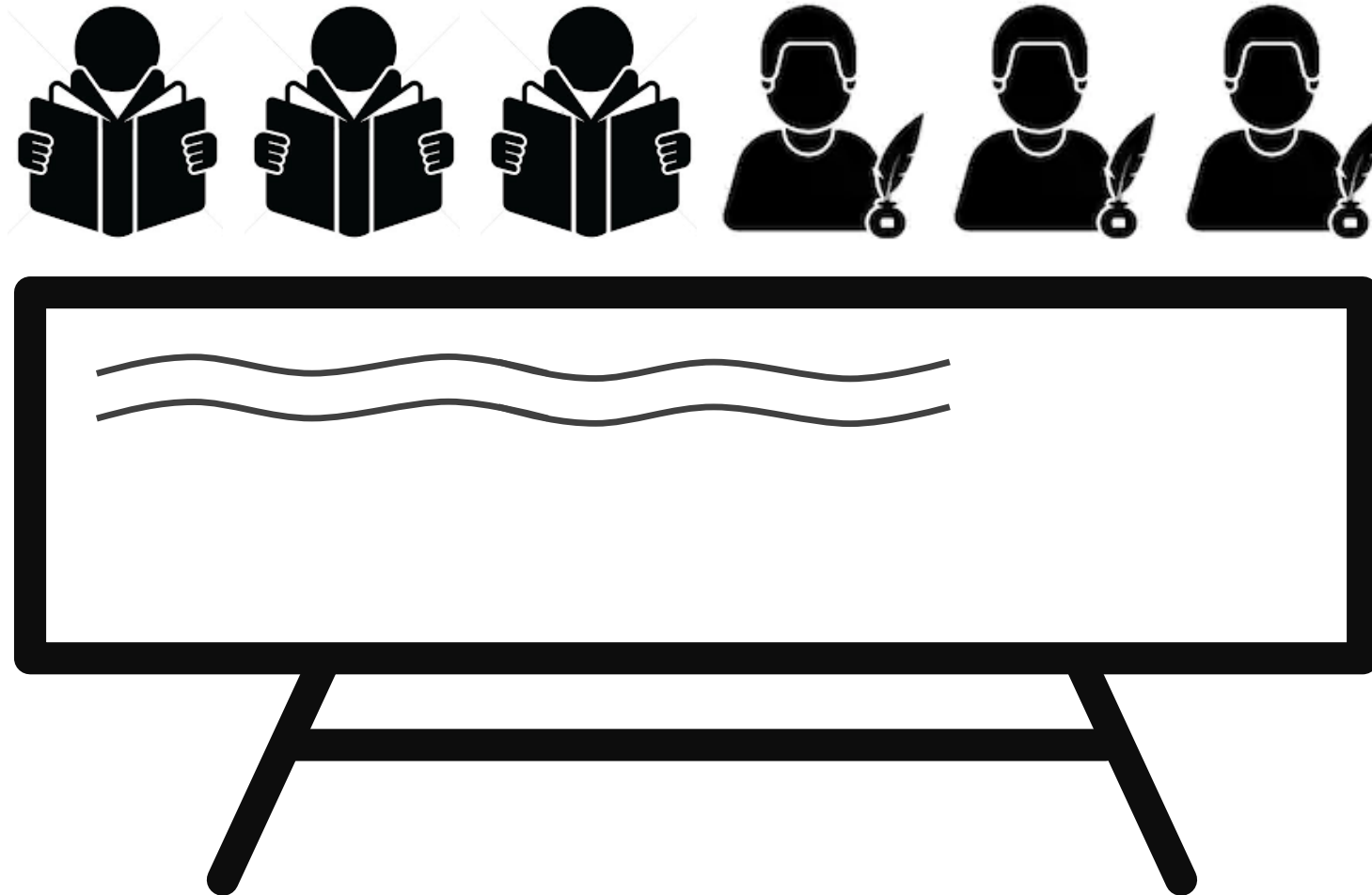
Produttore

```
while (true) {  
    // produce a new item  
    nFree.acquire();  
    lock.lock();  
    // store item  
    lock.unlock();  
    nItems.release();  
}
```

Consumatore

```
while (true) {  
    nItems.acquire();  
    lock.lock();  
    // remove item  
    lock.unlock();  
    nFree.release();  
    // consume the new item  
}
```

Lettori e Scrittori



Soluzione

```
public class SyncBoard<T> {  
    int nReaders = 0;  
    Lock lock = new Lock();  
    Semaphore empty = new Semaphore(1)  
    T board;  
    ...  
}
```

n. di lettori che stanno
leggendo

accesso in mutua esclusione
a nReaders

blocca gli scrittori quando ci
sono i lettori – blocca tutti gli
altri quando c'è uno scrittore

l'area condivisa

anche qui non si può usare un Lock
perché è acquisito da un thread e
rilasciato da un altro

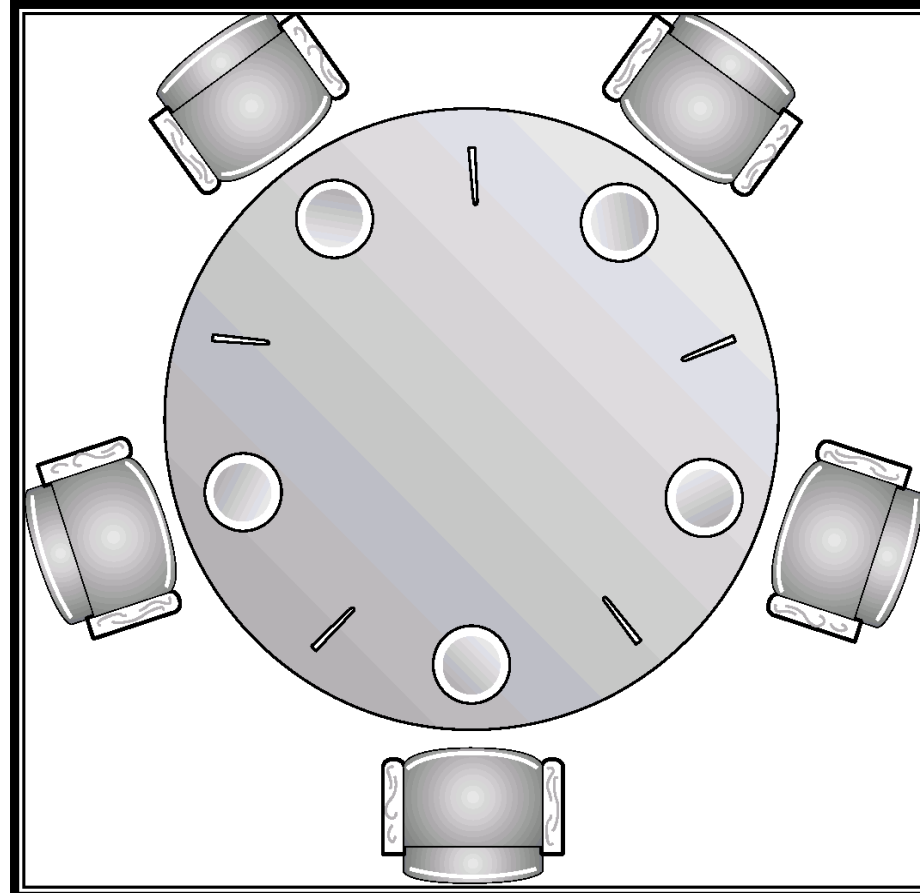
Scrittore

```
public void write(T msg) {  
    empty.acquire();  
    message = msg;  
    empty.release();  
}
```

Lettores

```
public T read() {  
    lock.lock();  
    nReaders += 1;  
    if (nReaders == 1)  
        empty.acquire();  
    lock.unlock();  
    T msg = message;  
    lock.lock();  
    nReaders -= 1;  
    if (nReaders == 0)  
        empty.release();  
    lock.unlock();  
    return msg;  
}
```

Cinque Filosofi



Problema

```
interface Table {  
    // philosopher k picks up forks  
    void getChopstick(int k);  
    // philosopher k releases forks  
    void putChopstick(int k);  
}
```

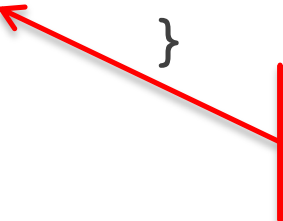
Filosofo *i*-esimo

```
while (true) {  
    think();  
    table.getChopstick(k); // wait for chopsticks  
    eat();  
    table.putChopstick(k); // release chopsticks  
}
```

Soluzione con Semafori

```
public class LockTable implements Table {  
    Lock[] chopstick = new Lock[N];
```

```
public void getChopstick(int k) {  
    // pick up left chopstick  
    chopstick[k].lock();  
    // pick up right chopstick  
    chopstick[(k+1)%N].lock();  
}  
  
public void putChopstick(int k) {  
    // put down left chopstick  
    chopstick[k].unlock();  
    // put down right chopstick  
    chopstick[(k+1)%N].unlock();  
}
```



Soddisfa mutua esclusione,
ma non il progresso



Sincronizzazione **I MONITOR**

I semafori forniscono un meccanismo potente e conciso per sincronizzazione e mutua esclusione. Sfortunatamente hanno diversi limiti:

- sono intrinsecamente “globali” e non strutturati
 - è difficile capire il loro comportamento guardando un singolo pezzo di codice
- sono soggetti a stallo o ad altri comportamenti scorretti
 - è facile dimenticare di aggiungere una singola chiamata cruciale verso l'alto o verso il basso

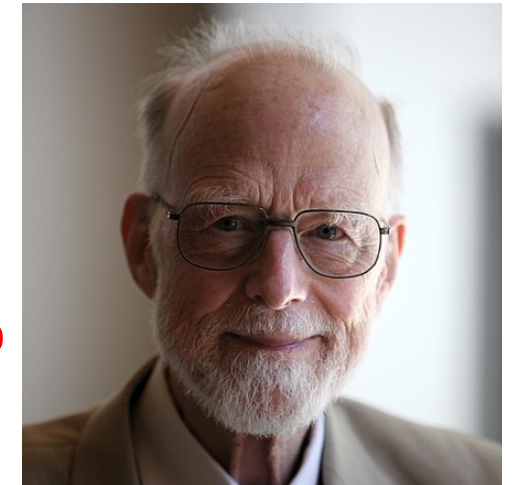
In sintesi i semafori sono una primitiva di sincronizzazione di basso livello.

I Monitor

I monitor forniscono un meccanismo di sincronizzazione strutturato costruito sopra i costrutti della programmazione orientati agli oggetti (classe, oggetto e incapsulamento).

- In una classe monitor:
 - gli attributi sono privati
 - i metodi pubblici vengono dichiarati eseguibili in mutua esclusione
- Un monitor è un oggetto che istanzia una classe di monitor.
- In che modo i monitor racchiudono i meccanismi di sincronizzazione:
 - gli attributi sono variabili condivise che tutti i thread in esecuzione sul monitor possono vedere e modificare
 - i metodi sono sezioni critiche, con la garanzia che in qualsiasi momento al massimo un solo thread può essere attivo su un monitor

C.A.R. Hoare

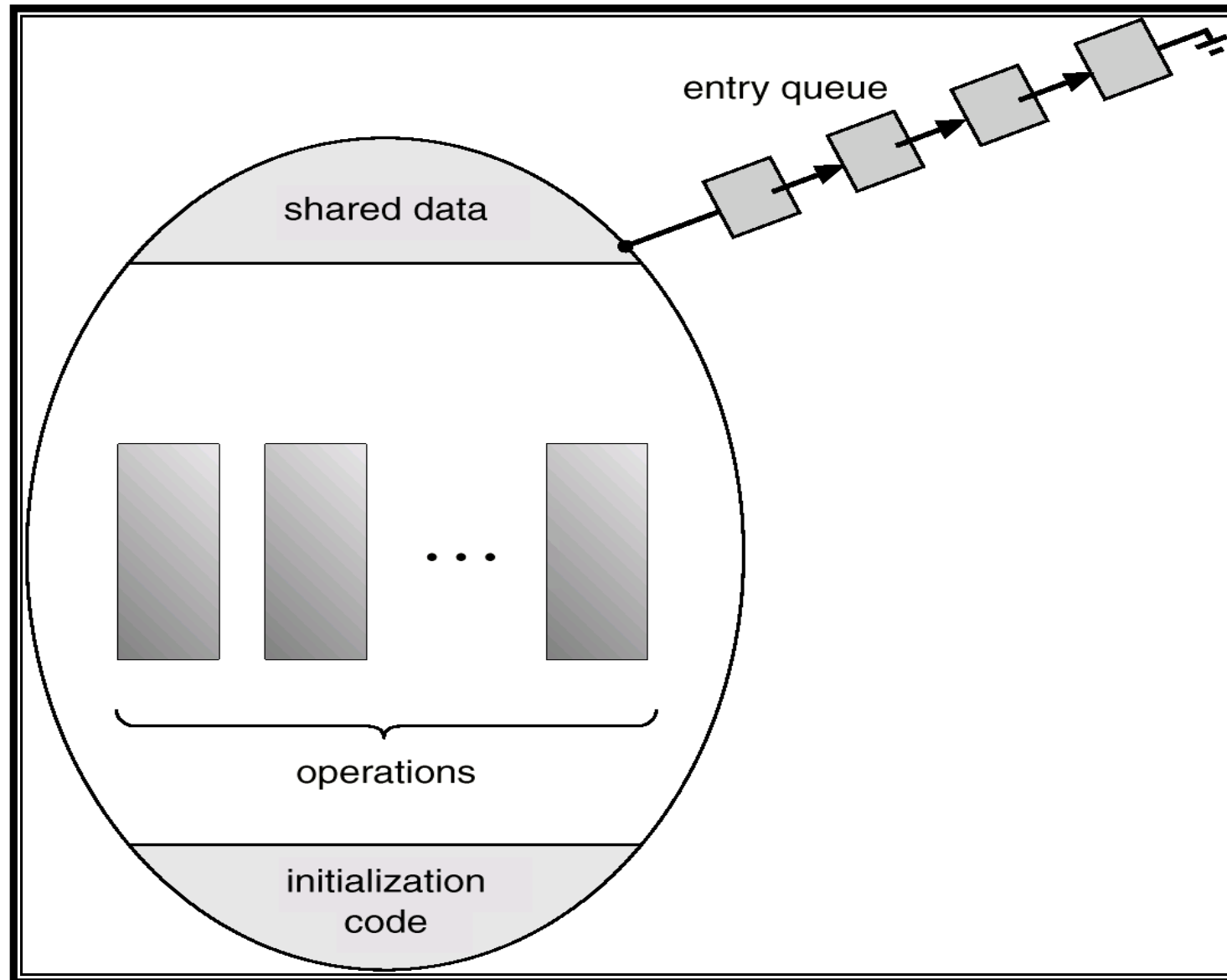


Turing Award 1980

Per Brinch Hansen



**IEEE Computer
Pioneer Award 2002**

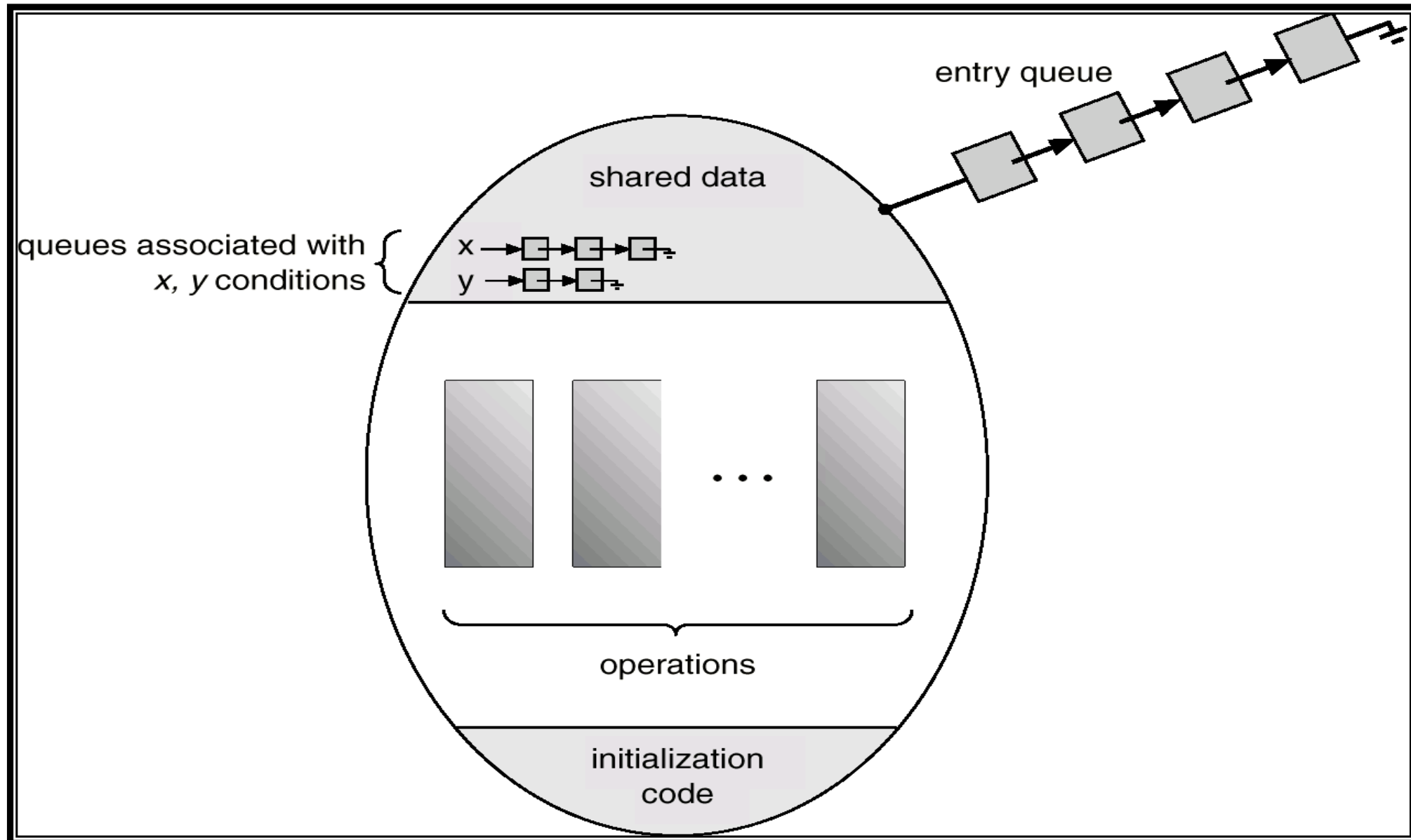


Per problemi di sincronizzazione più complessi della mutua esclusione i monitor mettono a disposizione anche le variabili di tipo **condition**.

- Le variabili di tipo condition permettono di accodare i processi in attesa
- una variabile condition è una istanza di una classe con interfaccia

```
interface Condition {  
    void wait();        // block until signal  
    void signal();      // signal to unblock  
}
```

- Un processo che invoca una `x.wait()` è sospeso (messo in coda) fino a quando un altro processo non invoca `x.signal()`;
- `x.signal` riavvia il primo processo in coda. Se nessun processo è sospeso, allora la `signal` non ha effetto.



Semantica delle Variabili Condition

- Se il processo P richiama `x.signal ()` e il processo Q è sospeso in `x.wait ()`, cosa dovrebbe succedere dopo?
 - Sia Q che P non possono essere eseguiti in parallelo. Se Q viene ripreso, allora P deve attendere
- Le opzioni includono
 - **Segnala e attendi** - P attende fino a quando Q lascia il monitor o attende un'altra condizione
 - **Segnala e continua** - Q attende che P lasci il monitor o attenda un'altra condizione
- Entrambe hanno vantaggi e svantaggi
 - Linguaggi di programmazione diversi adottano scelte diverse
- Il problema non si pone se `x.signal ()` è l'ultima istruzione di P.

Equivalenza Monitor-Semaforo

■ **Teorema**

Monitor e Semafori sono equivalenti dal punto di vista espressivo (si possono risolvere gli stessi problemi)

■ *Dimostrazione (suggerimento)*

- Un semaforo si può implementare usando un monitor.
- Un monitor si può implementare usando due semafori più un semaforo per ogni variabile condition.

- Java non implementa in modo completo i monitor, ma offre supporto per implementarle seguendo opportuni modelli di programmazione.
- In Java ci sono due serie di primitive simili ai monitor:
 - primitive **basate sul linguaggio**
 - primitive **basate sulle librerie**
- Le abbiamo già incontrate per implementare primitive di sincronizzazione più semplici.

Code di Accesso in Java

Ogni oggetto Java ha un lock implicito a cui si può accedere usando il modificatore **synchronized**.

Method Locking (synchronized method):

```
synchronized Type m() {  
    // the critical section  
    // is the whole method  
    // body  
}
```

Ogni chiamata ad m implicitamente:

1. fa una wait sul lock
2. esegue m
3. fa una signal sul lock

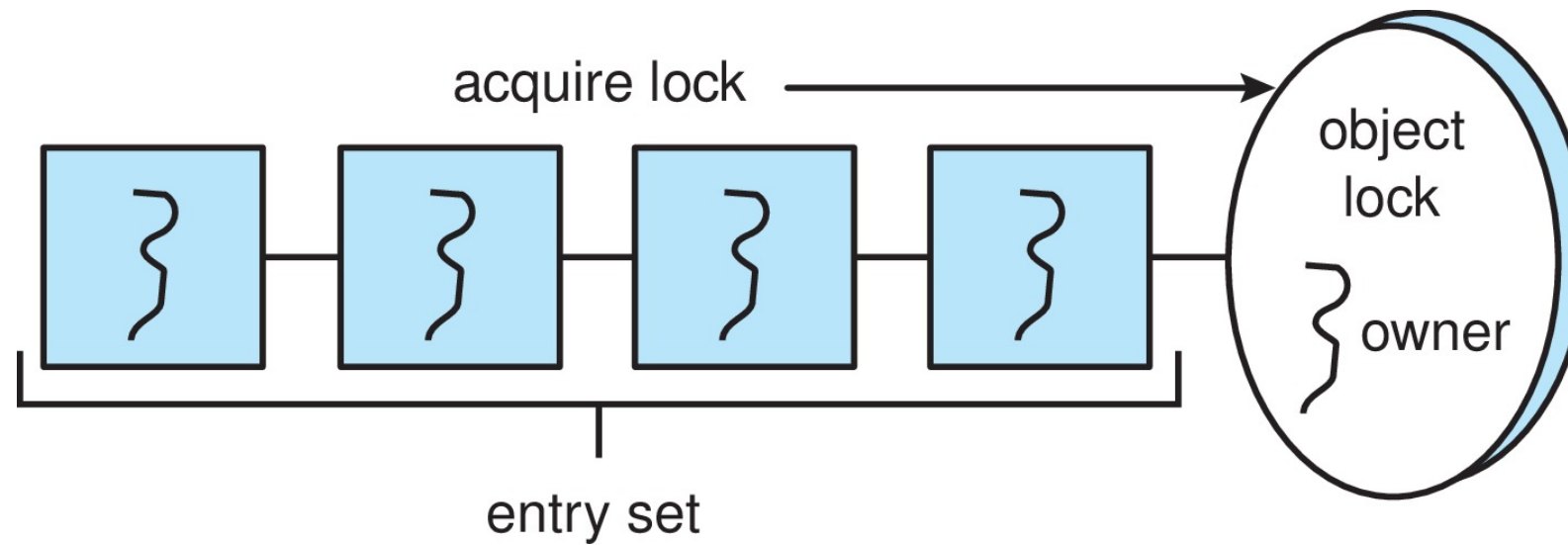
Block Locking (synchronized block):

```
synchronized(this) {  
    // the critical section  
    // is the block's content  
}
```

Ogni esecuzione del blocco implicitamente:

1. fa una wait sul lock
2. esegue il blocco
3. fa una signal sul lock

Code di Accesso in Java



Code di Accesso in Java: Un Esempio

Method Locking (synchronized method):

```
public class SyncCounter
implements Runnable
{
    private int counter = 0;

    public synchronized
    void run() {
        int tmp = counter;
        counter = tmp + 1;
    }
}
```

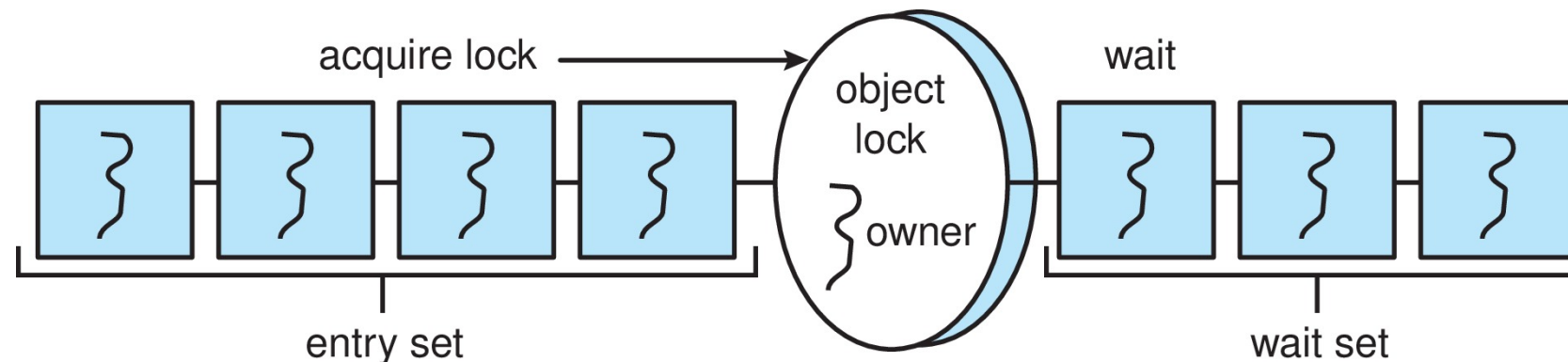
Block Locking (synchronized block):

```
public class SyncBlockCounter
implements Runnable
{
    private int counter = 0;

    public void run() {
        synchronized(this) {
            int tmp = counter;
            counter = tmp + 1;
        }
    }
}
```


Code di Attesa in Java

- Ogni oggetto Java eredita da Object tre metodi:
 - invocando **wait** () blocca il thread in esecuzione, in attesa di un segnale
 - invocando **notify** () sblocca un thread a caso tra quelli in attesa
 - invocando **notifyAll** () sblocca tutti i thread in attesa



Language-based Monitor

- Una classe JM può implementare un monitor M come segue:
 - ogni attributo in JM è privato
 - ogni metodo in JM è sincronizzato
 - garantendo l'esecuzione in modo atomico

```
monitor class M {  
    int x, y;  
    public void p()  
    { /* ... */ }  
    public int q()  
    { /* ... */ }  
}
```

```
class JM {  
    private int x, y;  
    public synchronized void p()  
    { /* ... */ }  
    public synchronized int q()  
    { /* ... */ }  
}
```

- Questo meccanismo **non garantisce l'equità** della coda di accesso associata al monitor
 - viene scelto dalla coda un thread in modo casuale

Language-based Condition

- Ogni language-based monitor include implicitamente una (sola) variabile condition con semantica di tipo **segnala e continua**:
 - invocando `wait()` blocca il thread in esecuzione, in attesa di un segnale
 - invocando `notify()` sblocca un thread a caso tra quelli in attesa del monitor
 - invocando `notifyAll()` sblocca tutti i thread in attesa del monitor

```
monitor class M {  
    int x; Condition y;  
    public void p()  
    { while (x < 0)  
        y.wait(); }  
    public int q()  
    { if (x > 0)  
        y.signal(); }  
}
```

```
class JM {  
    private int x;  
    public synchronized void p()  
    { while (x < 0)  
        wait(); }  
    public synchronized int q()  
    { if (x > 0)  
        notify(); }  
}
```

- Questo meccanismo **non garantisce l'equità** della coda di attesa
 - viene scelto dalla coda un thread in modo casuale

Library-based Monitor

- Una classe LM può implementare un monitor M usando variabili di lock:
 - aggiungere un lock come attributo privato
 - ogni metodo in LM usa in modo esplicito `lock()` e `unlock()`
 - garantendo l'esecuzione in modo atomico

```
monitor class M {  
  
    int x, y;  
    public void p()  
    { /* ... */ }  
  
}
```

```
class LM {  
    private final Lock monitor  
        = new ReentrantLock(true);  
    private int x, y;  
    public void p()  
    { monitor.lock();  
      /* ... */  
      monitor.unlock(); }  
}
```

- Questo meccanismo **garantisce l'equità** della coda di accesso associata al monitor

Library-based Condition

- Variabili condition con semantica di tipo **segnala e continua** possono essere generate da un lock.
 - I thread in attesa su una variabile condition che vengono “risvegliati” vengono messi nella coda di attesa del lock.
 - Per questo se il lock è fair anche le variabili condition saranno fair.

```
monitor class M {  
  
    Condition y;  
  
    int x;  
    // ...  
}
```

```
class LM {  
    private final Lock monitor  
        = new ReentrantLock(true);  
    private final Condition y  
        = monitor.newCondition();  
    private int x;  
    // ...  
}
```

Library-based Condition

- Variabili condition con semantica di tipo segnala e continua possono essere generate da un lock:

- invocando `await()` blocca il thread in esecuzione, in attesa di un segnale
- invocando `signal()` sblocca il primo thread in attesa sulla condition
- invocando `signalAll()` sblocca tutti i thread in attesa sulla condition

```
monitor class M {                                class LM {
    // ...                                         // ...
    public void p()                               public void p()
    {                                              {  monitor.lock();
        while (x < 0)                             while (x < 0)
            y.wait();                             y.await();
    }                                              monitor.unlock();  }
    // ...                                         // ...
}                                                  }
```

- Questo meccanismo **garantisce l'equità** della coda di attesa
 - anche `signalAll()` garantisce il rispetto dell'ordine di arrivo

Library-based Condition

- Variabili condition con semantica di tipo segnala e continua possono essere generate da un lock:

- invocando `await()` blocca il thread in esecuzione, in attesa di un segnale
- invocando `signal()` sblocca il primo thread in attesa sulla condition
- invocando `signalAll()` sblocca tutti i thread in attesa sulla condition

```
monitor class M {  
    // ...  
    public int q()  
    {  
        if (x > 0)  
            y.signal();  
    }  
}
```

```
class LM {  
    // ...  
    public int q()  
    {  
        monitor.lock();  
        if (x > 0)  
            y.signal();  
        monitor.unlock(); }  
}
```

- Questo meccanismo **garantisce l'equità** della coda di attesa
 - anche `signalAll()` garantisce il rispetto dell'ordine di arrivo

- Cosa succede se un thread ha una eccezione mentre possiede il controllo di un monitor?
- È importante che i programmi assicurino che in un caso del genere il thread lasci comunque il sistema in uno stato consistente rilasciando tutti i monitor che possiede:
 - nei language-based monitor, un thread che ha una eccezione mentre si trova in un metodo `synchronized` rilascia automaticamente il monitor
 - nei library-based monitor, utilizzare un blocco `finally` per rilasciare il blocco del monitor in caso di eccezione:

```
class LM {  
    private final Lock monitor = new ReentrantLock(true);  
    public void p() {  
        monitor.lock();  
        try { /* ... */ }  
        finally { monitor.unlock(); }  
    }  
}
```


Soluzione con Monitor

```
public class MonitorTable implements Table {  
    private enum State {THINKING , HUNGRY, EATING};  
    State[] state = new State[N];  
  
    private final Lock mutex = new ReentrantLock(true);  
    Condition[] self = new Condition[N];  
}
```

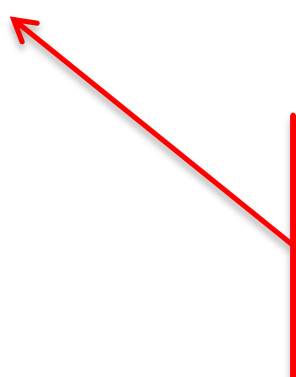
Soluzione con Monitor

```
public class MonitorTable implements Table {  
    public void MonitorTable() {  
        for (int i = 0; i < N; i++) {  
            state[i] = THINKING;  
            self[i] = mutex.newCondition();  
        }  
    }  
    private void test(int i) {  
        if ( (state[(i+4)%N] != EATING)  
            &&(state[i] == HUNGRY)  
            &&(state[(i+1)%N] != EATING) )  
        {  
            state[i] = EATING;  
            self[i].notify();  
        }  
    }  
}
```

Soluzione con Monitor

```
public class MonitorTable implements Table {  
    public void getChopstick(int i) {  
        mutex.lock();  
        try {  
            state[i] = HUNGRY;  
            test(i);  
            while (state[i] != EATING)  
            {  
                self[i].await();  
            }  
        } finally {  
            mutex.unlock();  
        }  
    }  
    public void putChopstick(int i) {  
        mutex.lock();  
        state[i] = THINKING;  
        test((i+4) % 5);  
        test((i+1) % 5);  
        mutex.unlock();  
    }  
}
```

Sis



Soddisfa mutua esclusione,
progresso, ma non l'attesa
limitata



Sincronizzazione

MODELLO A SCAMBIO DI MESSAGGI

Modello a Scambio di Messaggi

- I processi comunicano tra loro senza ricorrere alle variabili condivise



- non ci sono problemi di corsa critica
- Ci sono due messaggi:
 - send(message) – messaggi a dimensioni fisse o variabili
 - receive(message)
- Se P e Q desiderano comunicare, hanno bisogno:
 - Di stabilire un canale di comunicazione tra loro
 - Di scambiarsi i messaggi con le operazioni send/receive
- Realizzazione del canale di comunicazione
 - fisica (per es., memoria condivisa, hardware bus)
 - logica (per es., proprietà logiche)

Problemi di Realizzazione

- Come sono realizzati i canali?
- Un canale può essere associato a più di due processi?
- Quanti canali ci possono essere per ogni coppia di processi?
- Quale è la capacità di un canale?
- Il tipo di canale può permettere messaggi di dimensione fissa o variabile?
- Il canale è unidirezionale o bidirezionale?



Soluzione con Scambio di Messaggi

```
public class Message {  
    public static final int BUFFER_SIZE = 10;  
    private Type value;  
  
    public void send(Thread t) { /* ... */}  
    public void receive(Thread t) { /* ... */}  
    public void setValue(Type value) { /* ... */}  
    public Type getValue() { /* ... */}  
}
```

Soluzione con Scambio di Messaggi

```
public void producer() {  
    Message msg;  
    Type v;  
  
    while (true) {  
        v = produce_item();  
        msg.receive(consumer);  
        msg.setValue(v);  
        msg.send(consumer);  
    }  
}
```

```
private void consumer() {  
    Message msg;  
    Type v;  
  
    for (i=0; i<BUFFER_SIZE; i++)  
        msg.send(producer);  
    while (true) {  
        msg.receive(producer);  
        v = msg.getValue();  
        msg.send(producer);  
        consumer_item(v);  
    }  
}
```

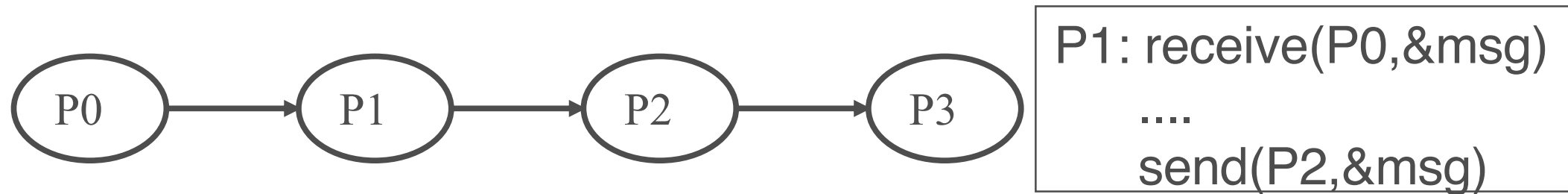



Designazione di mittente/destinatario

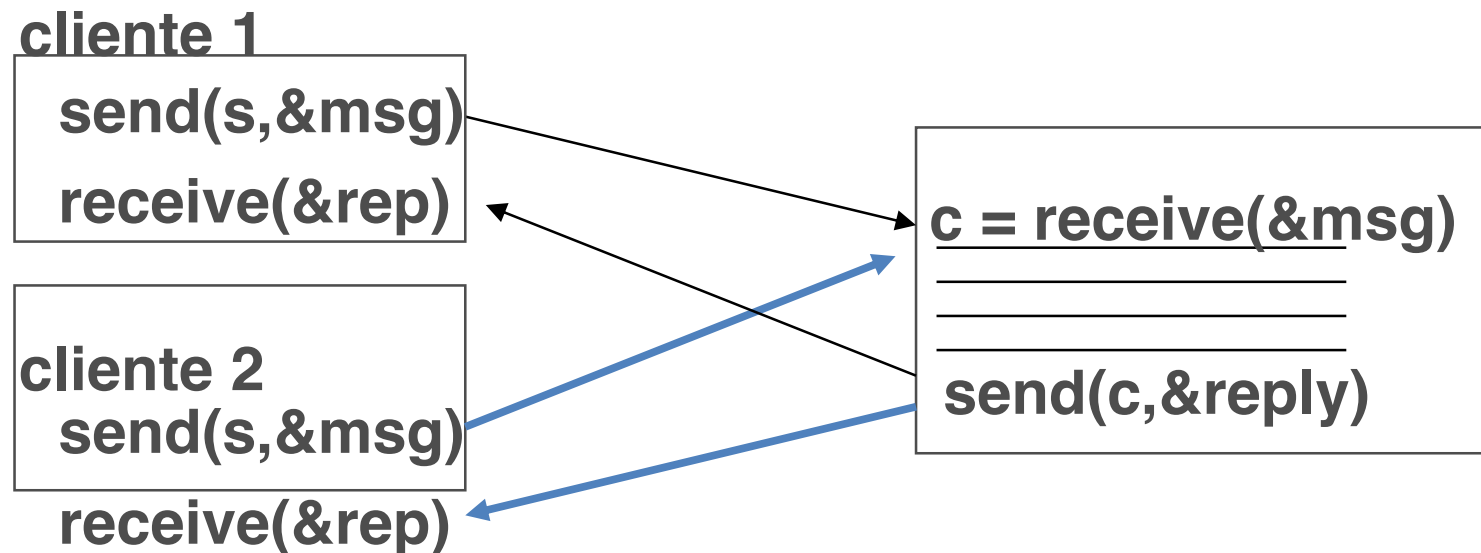
- Definizione: Canale – collegamento logico tramite il quale due processi comunicano
- Comunicazione simmetrica: si usano direttamente i nomi dei processi coinvolti per denotare un canale
- Comunicazione asimmetrica: si ha quando il processo mittente nomina esplicitamente il destinatario, ma quest'ultimo invece non esprime il nome del processo da cui vuole ricevere

Esempi di modelli di comunicazione simmetrica/asimmetrica

- Comunicazione simmetrica: la pipeline



- Comunicazione asimmetrica: modello cliente servitore, schema molti (clienti) a uno (servitore)



- Ogni processo deve nominare esplicitamente il processo con cui vuole comunicare:
 - `send (P, message)` – invia un messaggio al processo P
 - `receive(Q, message)` – ricevi un messaggio dal processo Q
- Proprietà del canale di comunicazione
 - I canali sono stabiliti automaticamente.
 - Ad ogni canale è associata una coppia di processi comunicanti.
 - Per ogni coppia esiste un solo canale.
 - I canali possono essere unidirezionali o (di solito) bidirezionali.

- I messaggi sono diretti verso / ricevuti da mailbox (anche dette porte).
 - Ogni mailbox ha un unico identificatore.
 - Processi possono comunicare solo se condividono la mailbox.
- Proprietà del canale di comunicazione
 - Viene stabilito un canale solo se i processi condividono la mailbox
 - Un canale può essere associato a diversi processi.
 - Ogni coppia di processi può condividere diversi canali di comunicazione.
 - I canali possono essere unidirezionali o bidirezionali.

■ Operazioni

- Creare una nuova mailbox
- Inviare e ricevere messaggi attraverso la mailbox
- Distruggere una mailbox

■ Primitive:

- `send(A, message)` – invia un messaggio alla mailbox A
- `receive(A, message)` – ricevi un messaggio dalla mailbox A

- **Condivisione di mailbox**
 - P1, P2, e P3 condividono la stessa mailbox A.
 - P1, invia un messaggio; P2 e P3 ricevono.
 - Chi prende il messaggio?
- **Soluzione**
 - Permettere ad un canale di essere associato solo a due processi alla volta.
 - Permettere a solo un processo alla volta di effettuare l'operazione di ricevere messaggi.
 - Permettere al sistema di selezionare arbitrariamente il ricevente. Il sistema deve poi notificare al mittente chi ha ricevuto il messaggio.

- La coda di messaggi associata ad un canale di comunicazione può essere realizzata in uno dei seguenti tre modi:
 1. Capacità Zero– 0 messaggi
Il mittente deve attendere che il ricevente legga il messaggio prima di inviare un nuovo messaggio (rendez-vous).
 2. Capacità Limitata – lunghezza finita di n messaggi
Il mittente deve attendere solo quando il canale è pieno.
 3. Capacità Illimitata – lunghezza infinita
Il mittente non deve attendere mai..

Primitive di sincronizzazione

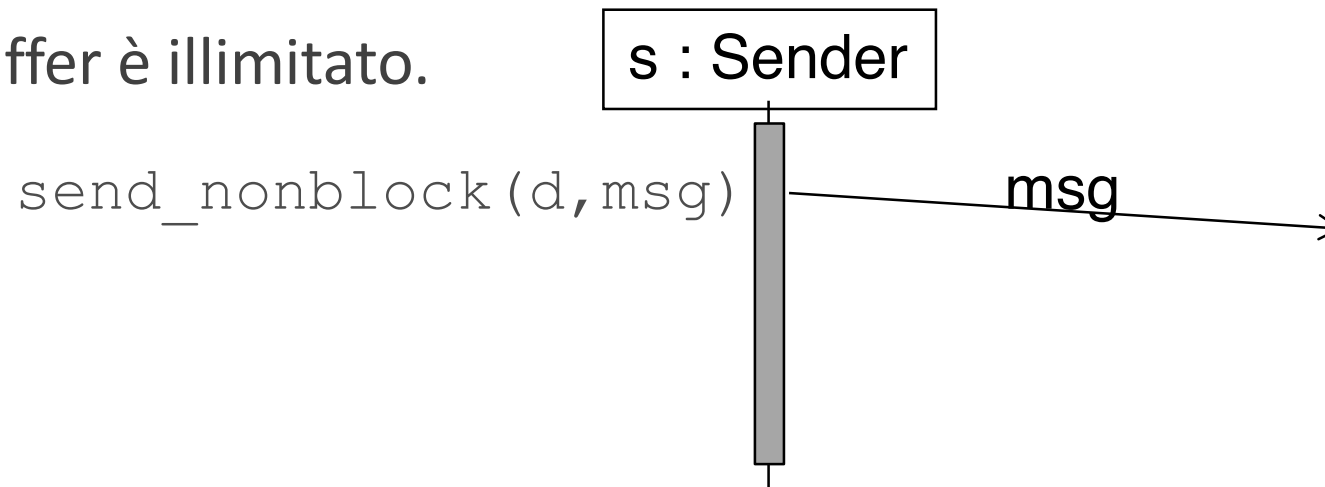
- Tipi di send
 - Send non bloccante
 - Send bloccante con buffer limitato
 - Send bloccante

- Tipi di receive
 - Receive non bloccante
 - Receive bloccante

Send non bloccante

Il mittente prosegue la sua esecuzione immediatamente dopo che il messaggio è stato inviato, senza attendere che sia stato ricevuto.

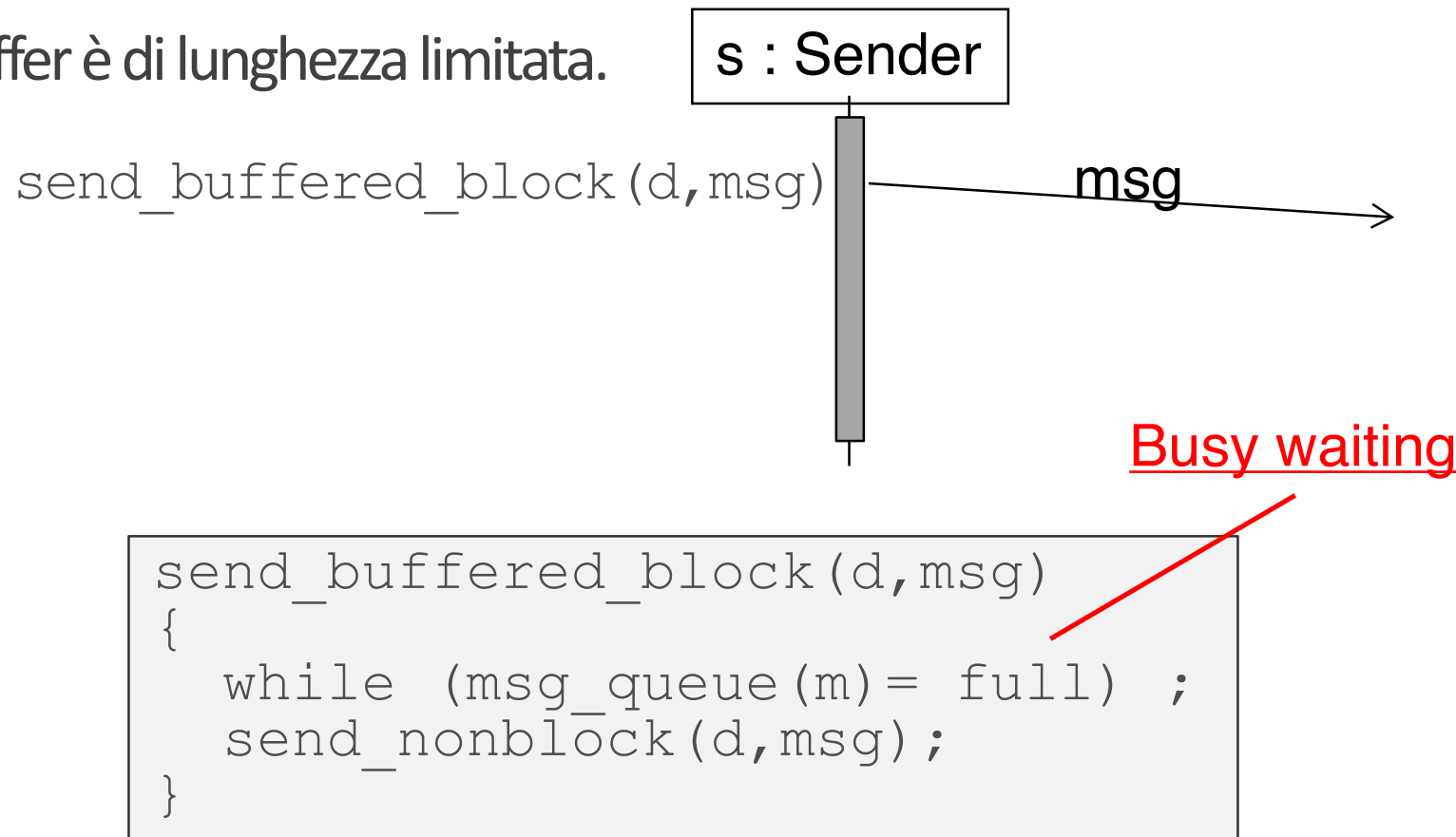
- Il mittente non attende mai la ricezione del messaggio da parte del destinatario
- Il buffer è illimitato.



Send bloccante con buffer limitato

Il mittente prosegue la sua esecuzione immediatamente dopo che il messaggio è stato inviato, senza attendere che sia stato ricevuto, a meno che il buffer non sia pieno.

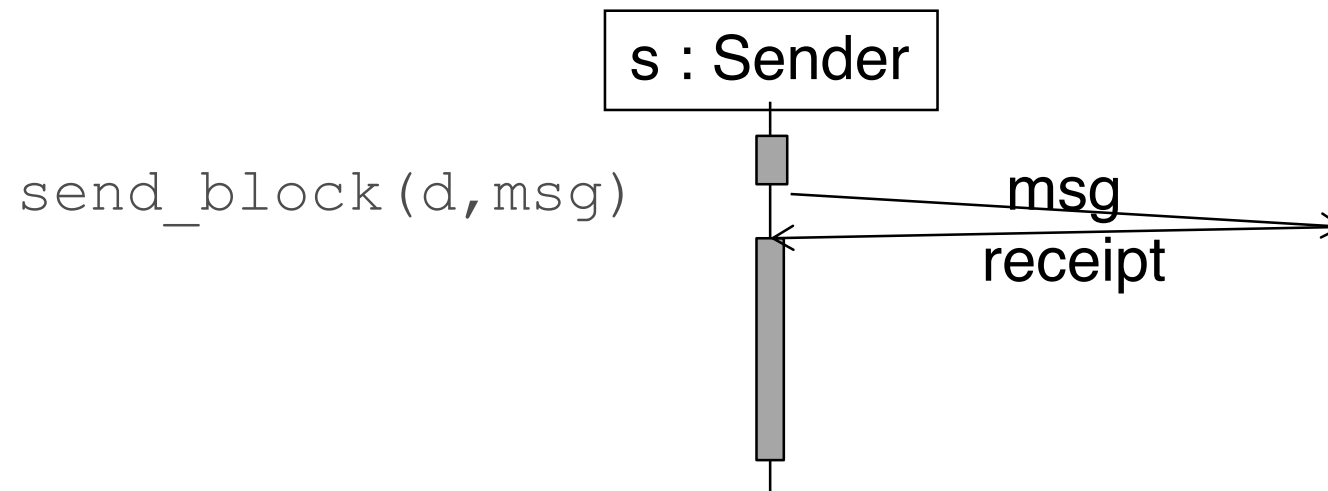
- Il mittente non attende la ricezione del messaggio da parte del destinatario a meno che il buffer non sia pieno
- Il buffer è di lunghezza limitata.



Send bloccante

Il mittente attende che il messaggio sia stato ricevuto dal destinatario.

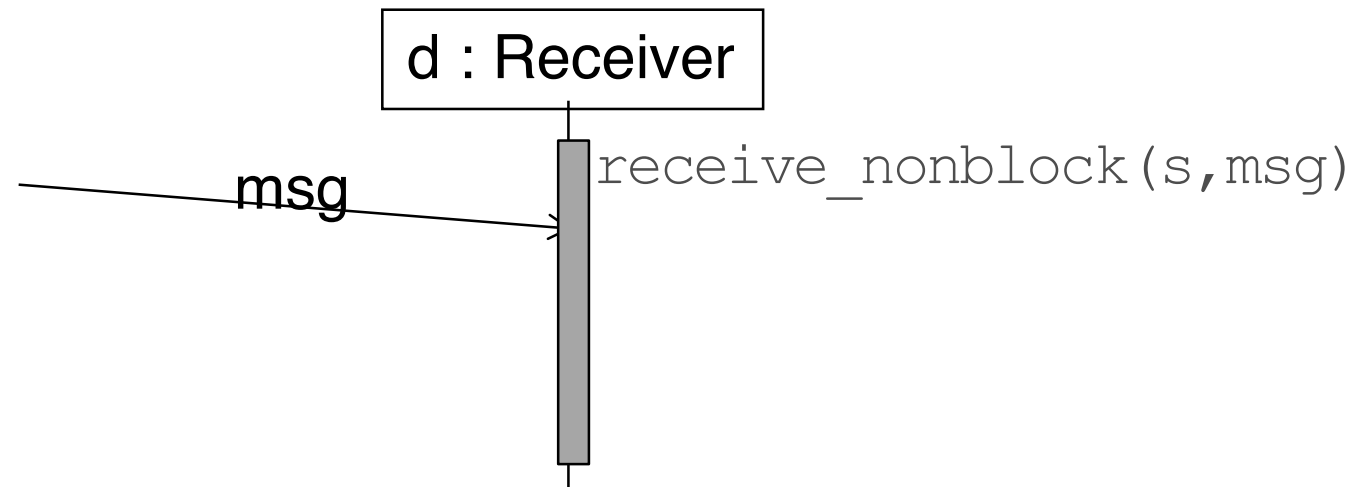
- Il mittente attende la ricezione del messaggio da parte del destinatario
- Il buffer è di lunghezza nulla.



```
send_block(d, msg)
{
    send_nonblock(d, msg);
    receive_block(d, receipt);
}
```

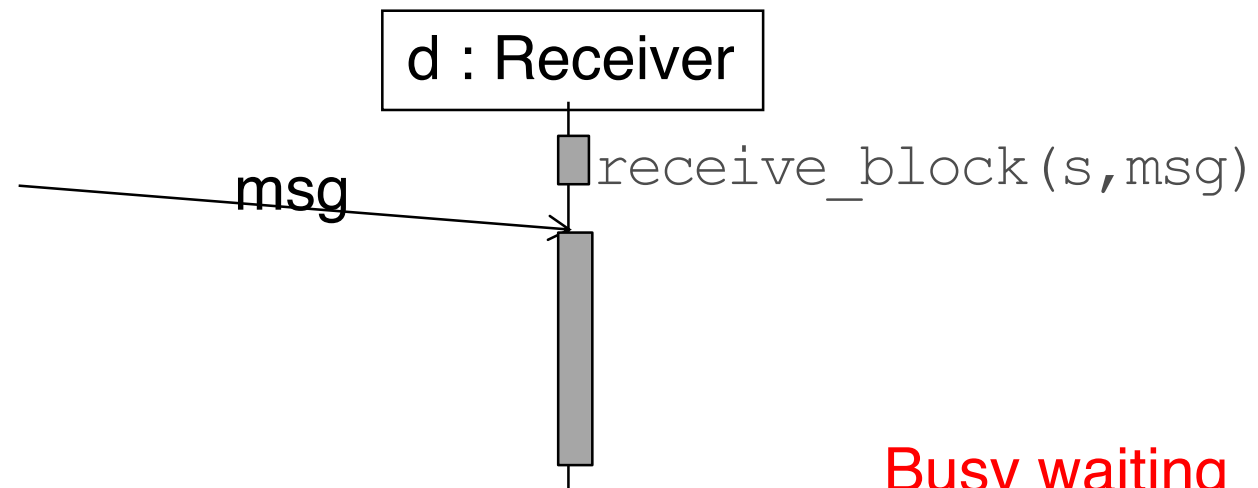
Receive non bloccante

Il destinatario che esegue una receive non rimane bloccato. Di solito in questo caso esiste anche una funzione che permette di verificare quando il buffer indicato in una receive precedente contiene effettivamente un messaggio.



Receive bloccante

Questa è la scelta più comune: il destinatario che esegue una receive rimane bloccato (stato waiting) finché non è disponibile un messaggio per lui. Quando il messaggio arriva ad un processo bloccato su una receive, quest'ultimo viene riattivato (stato ready).



Busy waiting

```
receive_block(m, msg)
{
    while (msg_queue(m) == empty) ;
    receive_nonblock(m, msg);
}
```

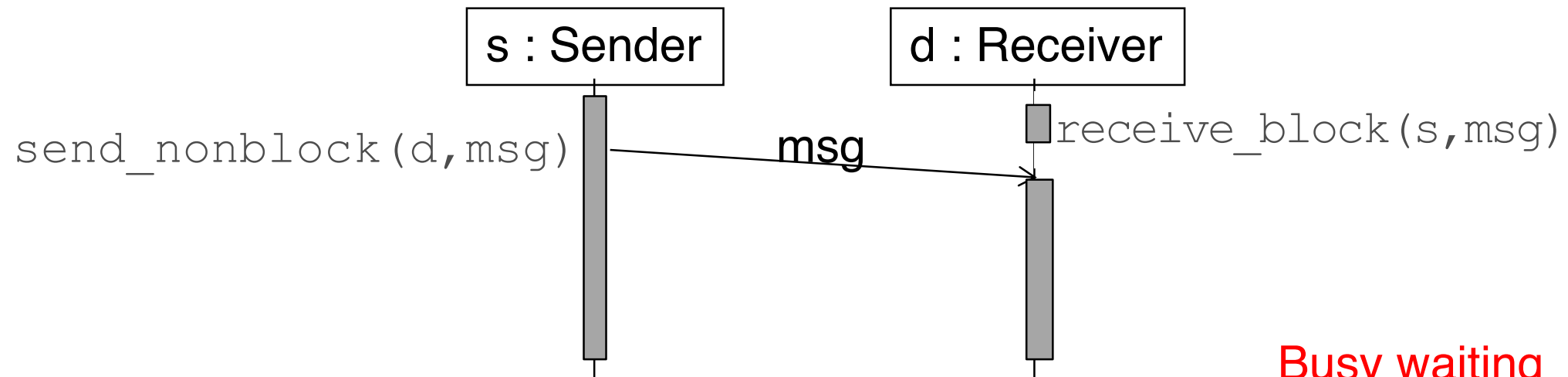


Meccanismi di sincronizzazione

- Scambio di messaggi Asincrono
- Scambio di messaggi Sincrono
- Invocazione remota

Scambio di Messaggi Asincrono

- Send non bloccante o bloccante con buffer limitato
- Receive bloccante



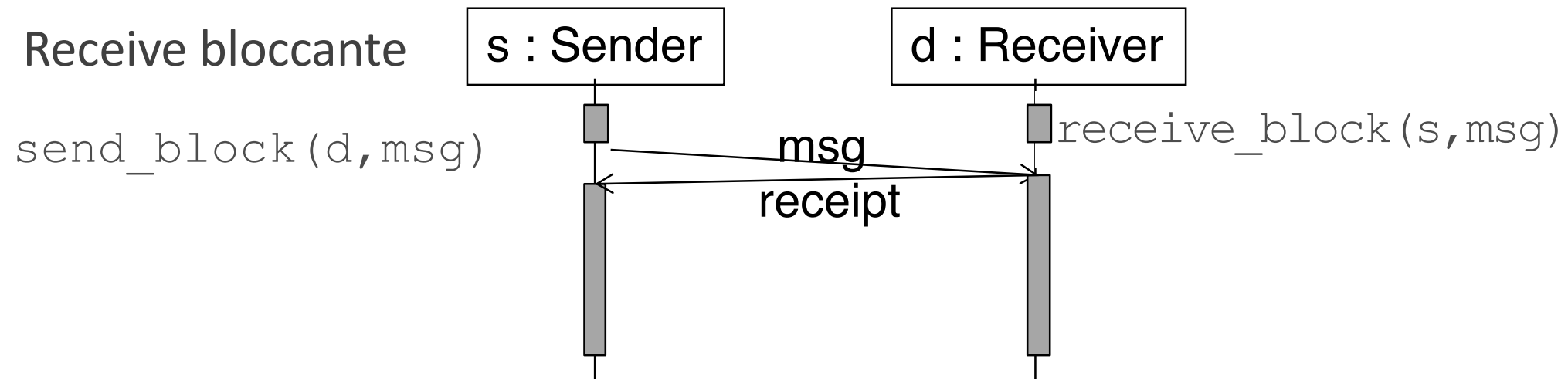
Busy waiting

```
receive_block(m,msg)
{
    while (msg_queue(m)=empty) ;
    receive_nonblock(m,msg);
}
```

Scambio di Messaggi Sincrono

Il passaggio del messaggio dal mittente al destinatario costituisce un punto di sincronizzazione (rendez-vous). Il messaggio ricevuto contiene informazioni sullo stato attuale del mittente.

- Send bloccante
- Receive bloccante



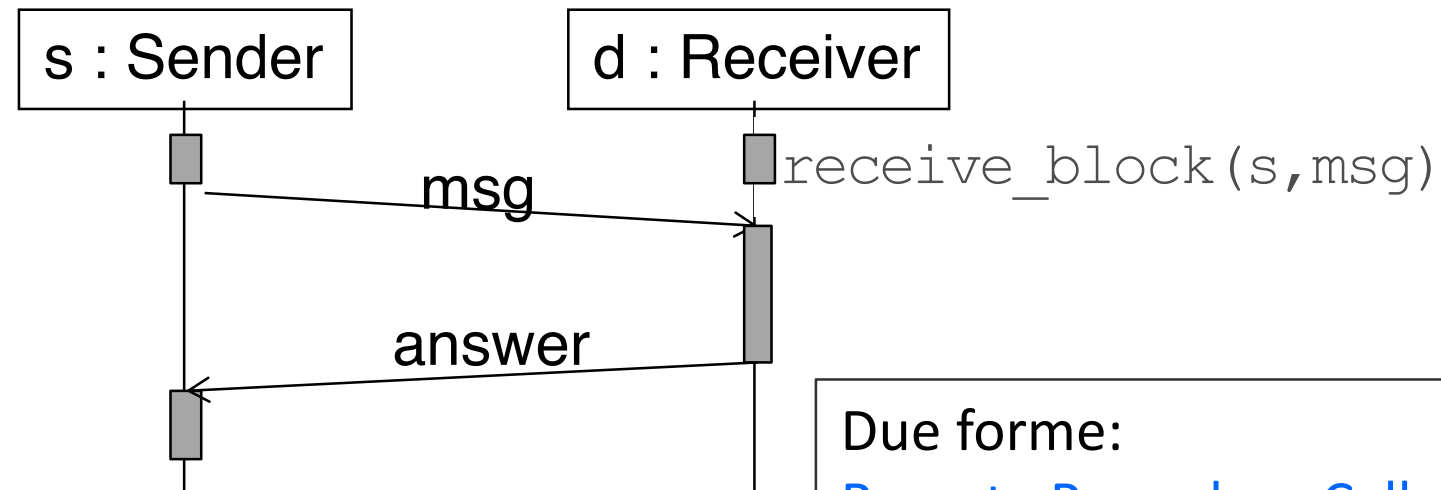
```
send_block(d, msg)
{
    send_nonblock(d, msg);
    receive_block(d, receipt);
}
```


Invocazione Remota

Il processo che esegue la chiamata è un cliente, mentre la procedura viene eseguita da un processo servitore remoto. Il cliente rimane sospeso fino a quando il servitore non ha terminato il servizio (rendez-vous esteso).

- Send bloccante
- Receive bloccante

`call(d, msg)`



```
call(d, msg)
{
    send_nonblock(d, msg);
    receive_block(d, answer);
}
```

Due forme:

Remote Procedure Call

- Viene creato un nuovo processo per ogni call

Extended Rendezvous

- Riceve il messaggio un processo già esistente

- Dato che nel modello a scambio di messaggi non c'è memoria condivisa, come possiamo implementare applicazioni che risulta naturale organizzare logicamente in più processi che devono poter accedere a strutture dati comuni?
- La soluzione consiste nell'introdurre particolari processi “gestori” delle strutture dati che si vogliono condividere, ed inviare ogni richiesta di lettura/modifica di tali strutture a questi ultimi.